



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA



CORSO DI GRAFICA E INTERATTIVITA'

2024/2025

Hands on Game Engine

Lezione 2: Creare una scena in Unity3D

Docenti:

Prof. Andrea F. Abate

Dott.ssa Lucia Cimmino



How to manage the 3D space

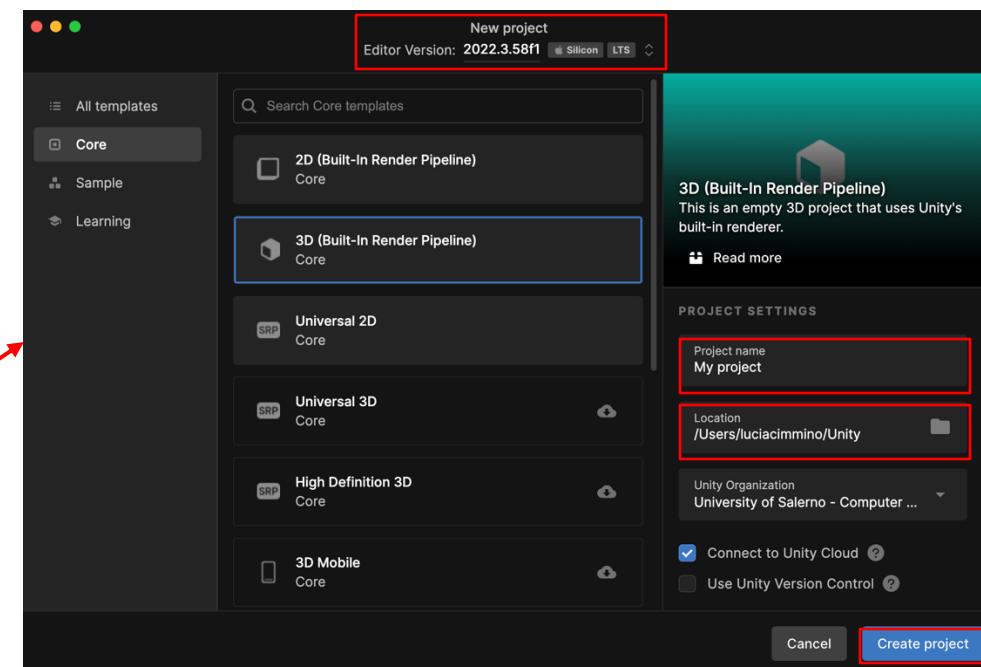
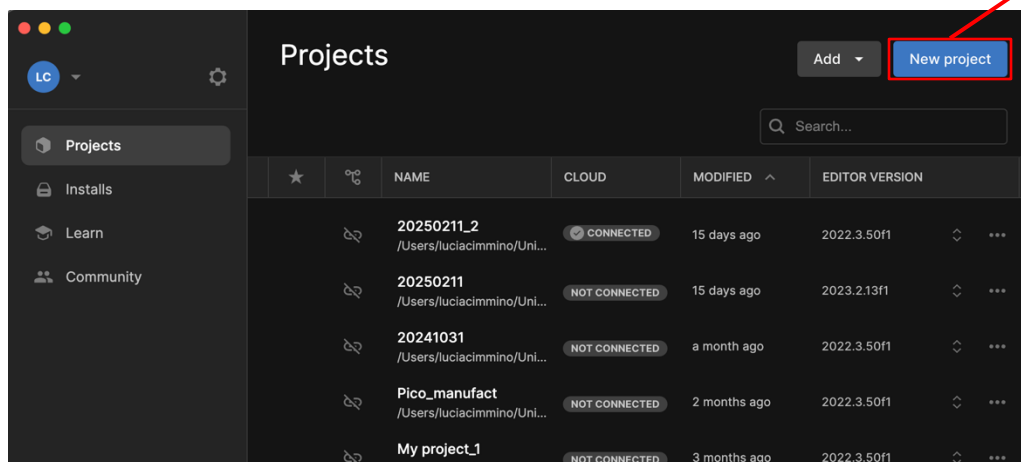
Let's Start!

Where do I start from?

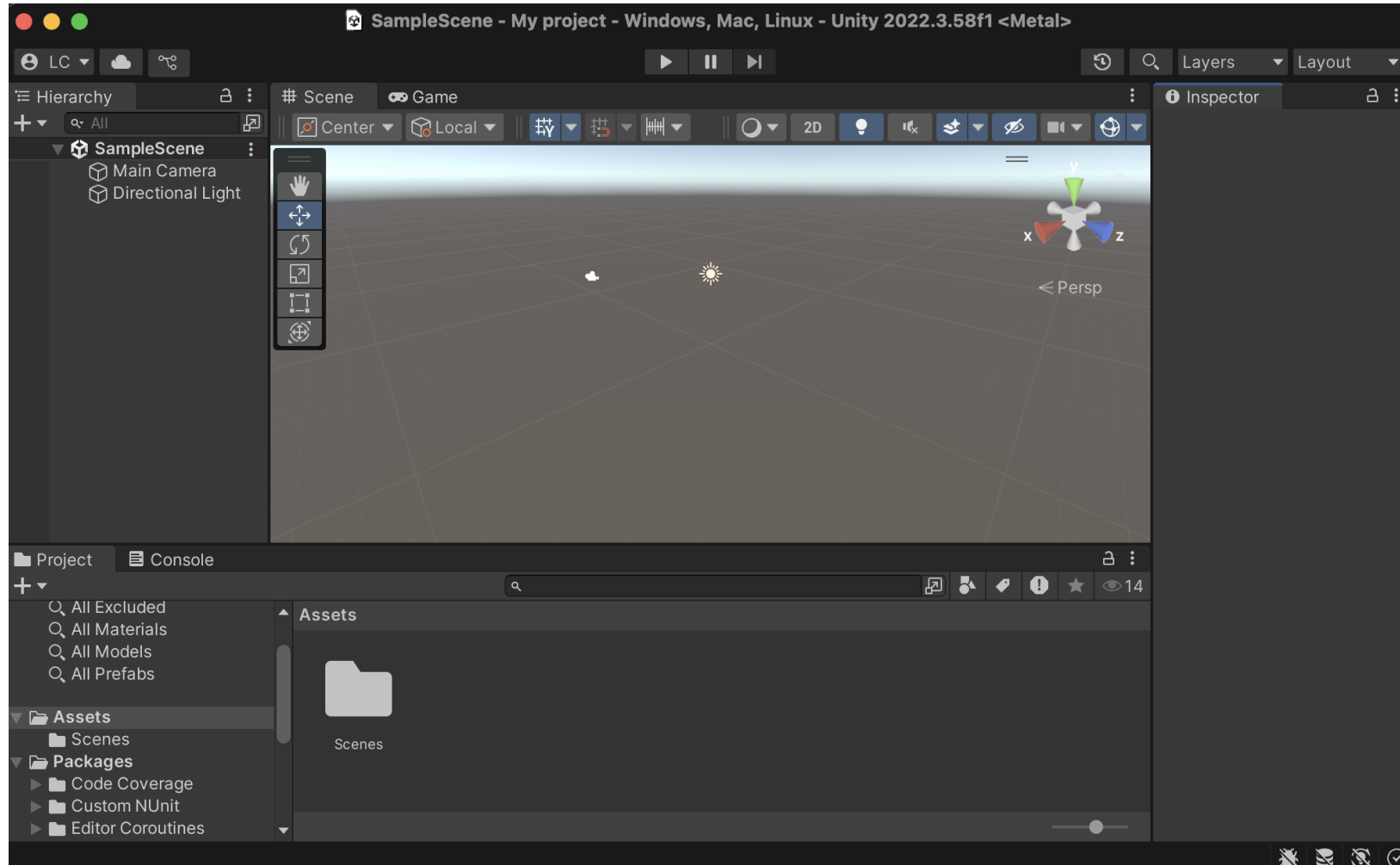
- Before you start programming anything, you always want to pause and ask yourself, “***So what am I building here?***”
- **Game design** is a huge topic unto itself, with many impressively large books focused on how to design a game.
- During this course we will explore the basis of graphic application programming to learn how to manage 3D scene and how to build what we have in mind.

Create a project

- Open Unity Hub and go to the Projects tab.
- Click "New Project".
- Select the 3D (core) Template.
- Check the Unity version at the top of the interface.
- Name your project.
- Choose the save location for your project.
- Click "Create Project" to start working in Unity.



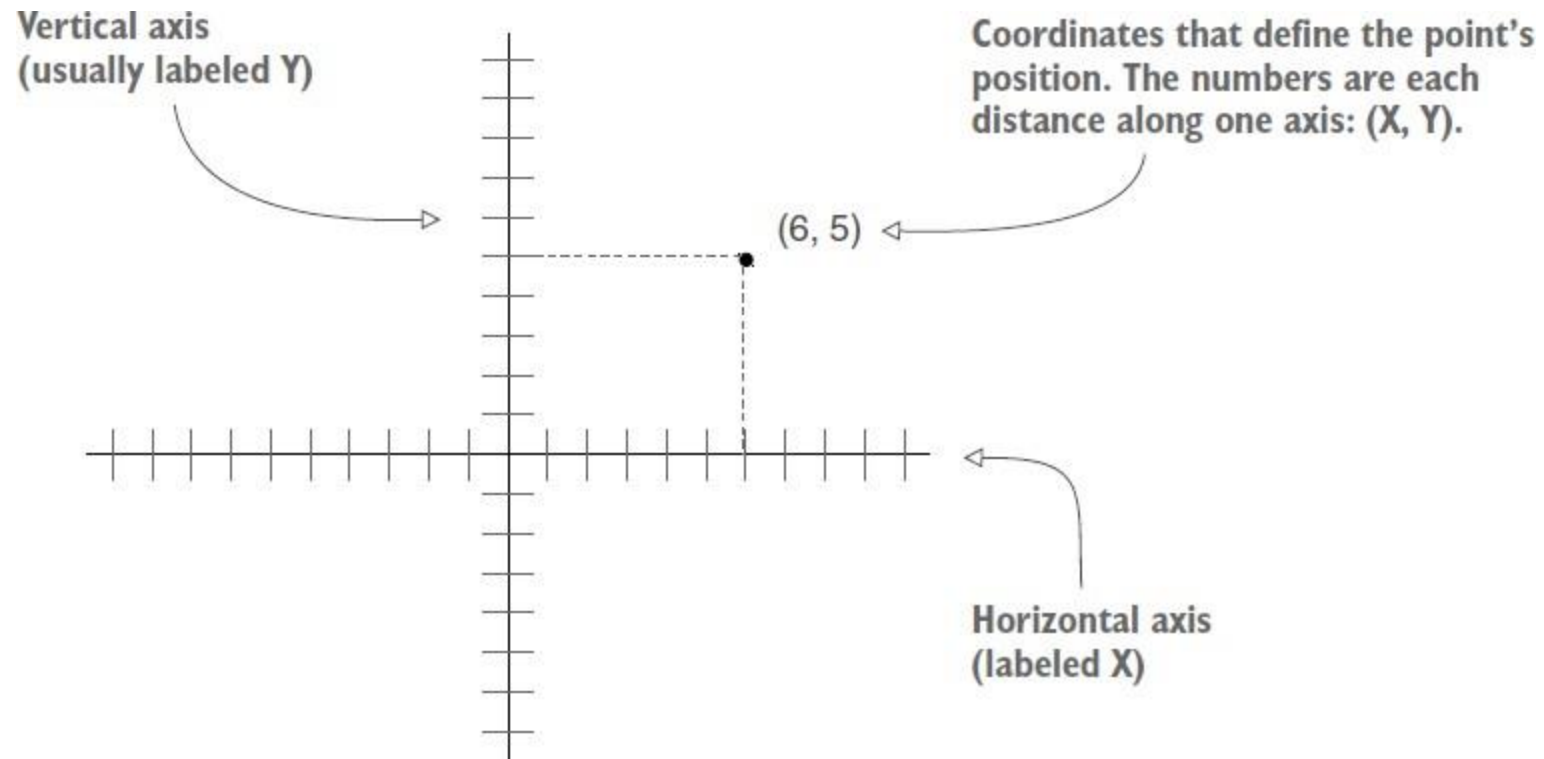
Our Unity Empty Project



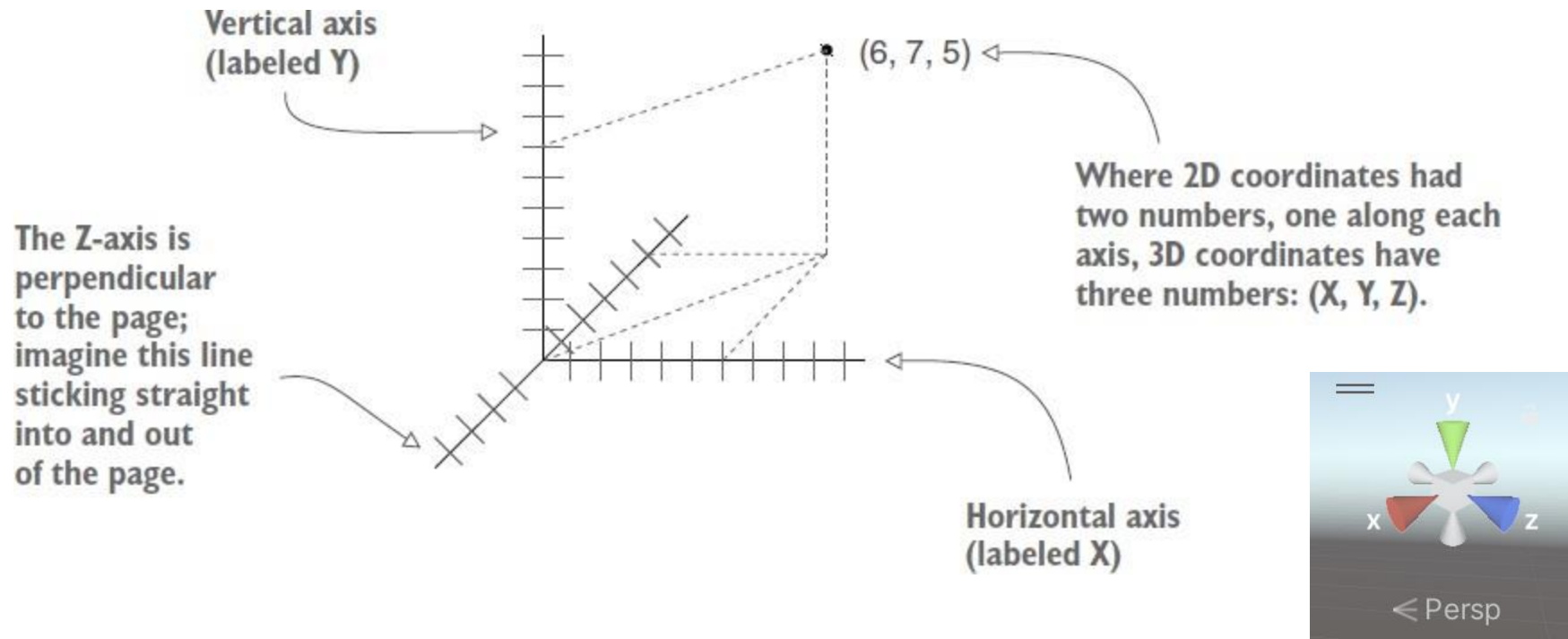
Understanding 3D Space

- 3D space is a mathematical representation where objects have width (X), height (Y), and depth (Z).
- Every position and movement in a 3D simulation or game engine is defined using these three axes.
- Understanding how objects are placed and manipulated along these axes is fundamental for creating 3D worlds.

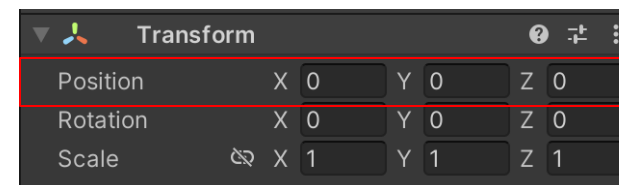
The 2D Space



The 3D Space

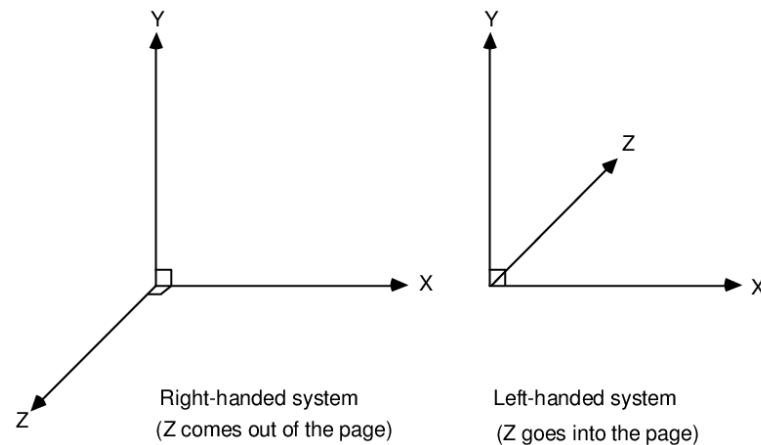


- In Unity's Scene view you can see the three axes displayed.
- In the Inspector you can type in the three numbers to position an object.

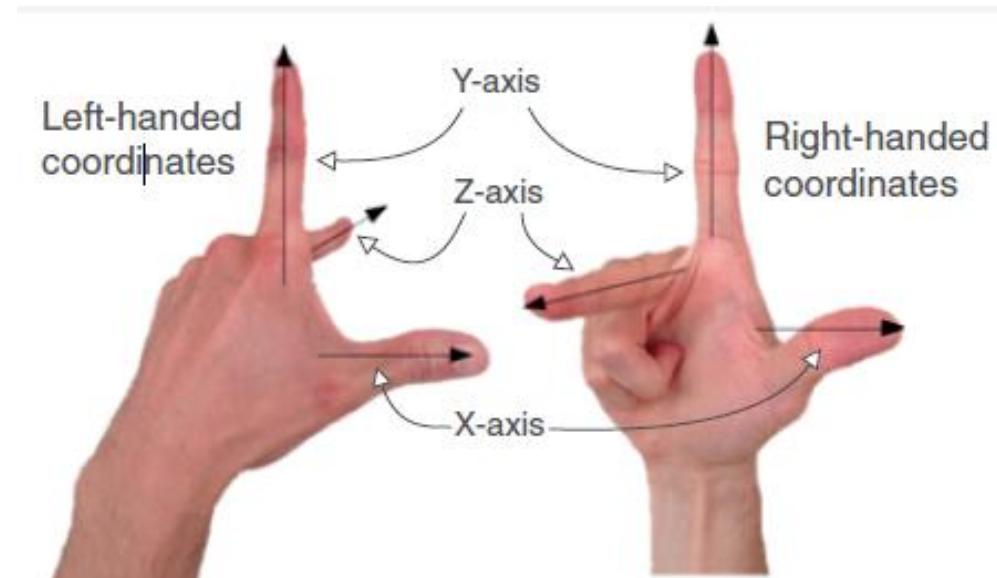


Positive/negative sign of axes

- The positive and negative direction of each axis is **arbitrary**, and the coordinates still work no matter which direction the axes point.
- You simply need to stay consistent within a given 3D graphics tool (animation tool, game development tool, etc.).



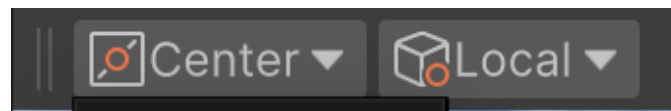
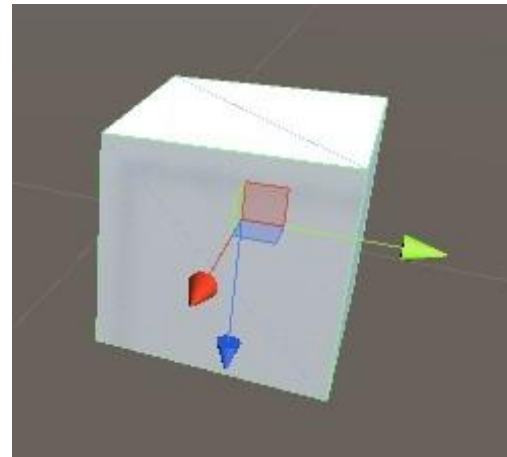
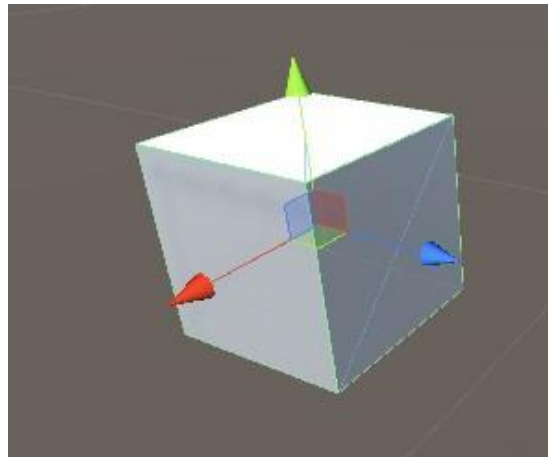
 Unity uses a **left-handed** coordinate system, as do many (but not all!!!) 3D art applications.



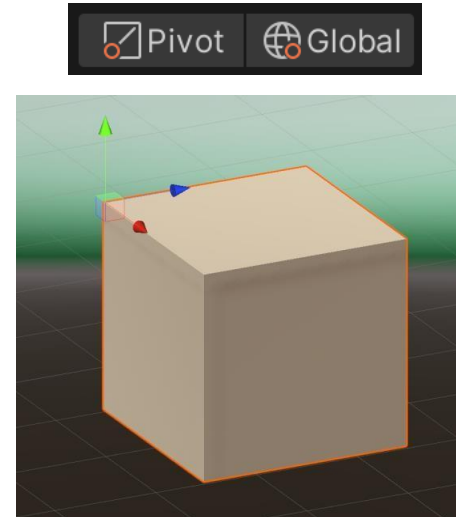
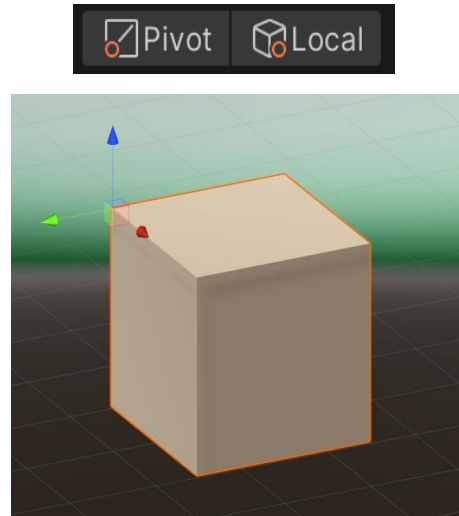
Why is the coordinate system important to know?



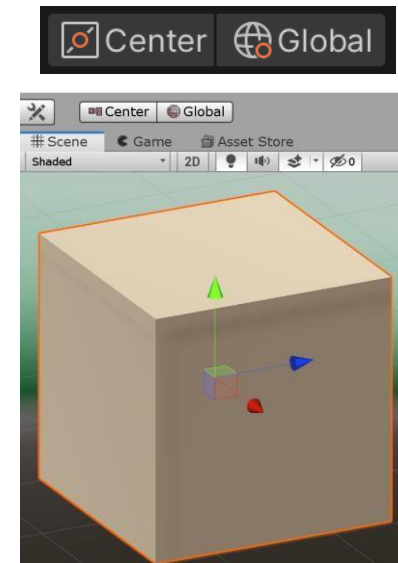
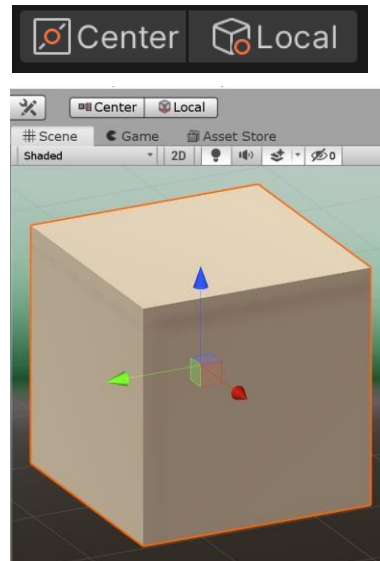
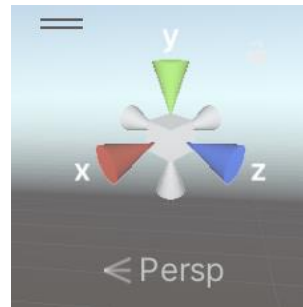
- Interoperability!
- When using models and projects built in another 3D tools, we can get very confused.



Let's understand coordinate system



Unity coordinates



Scene Creation



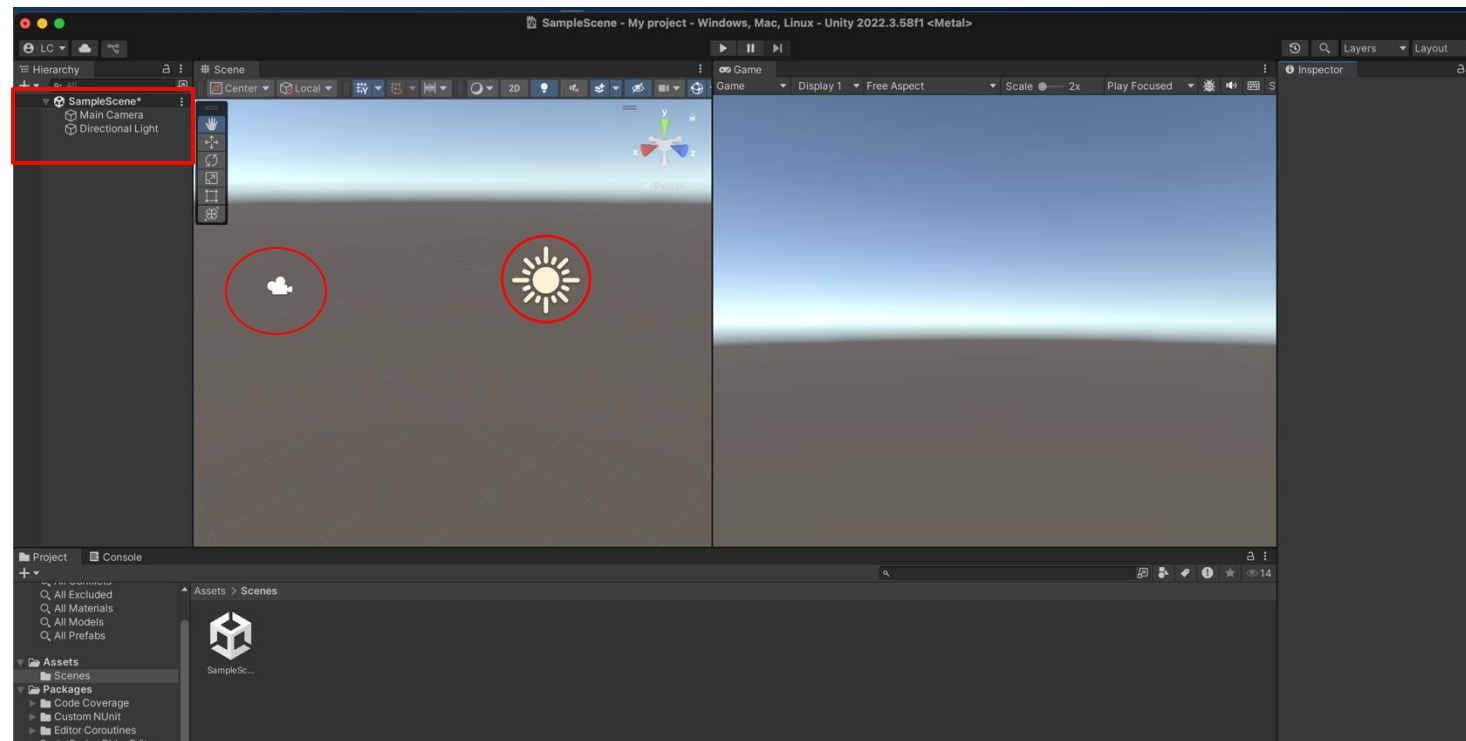
Our Unity Empty Project



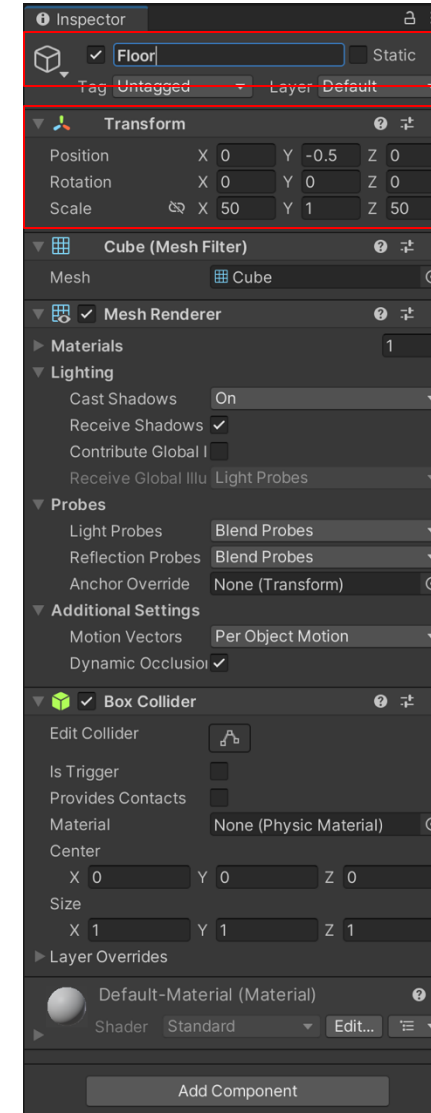
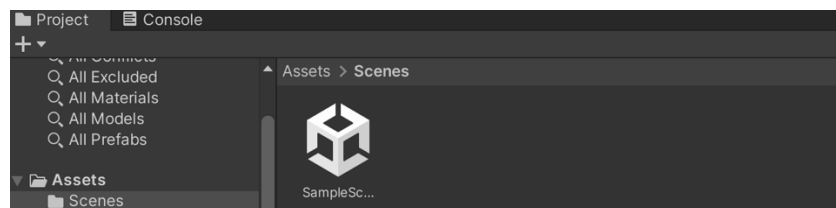
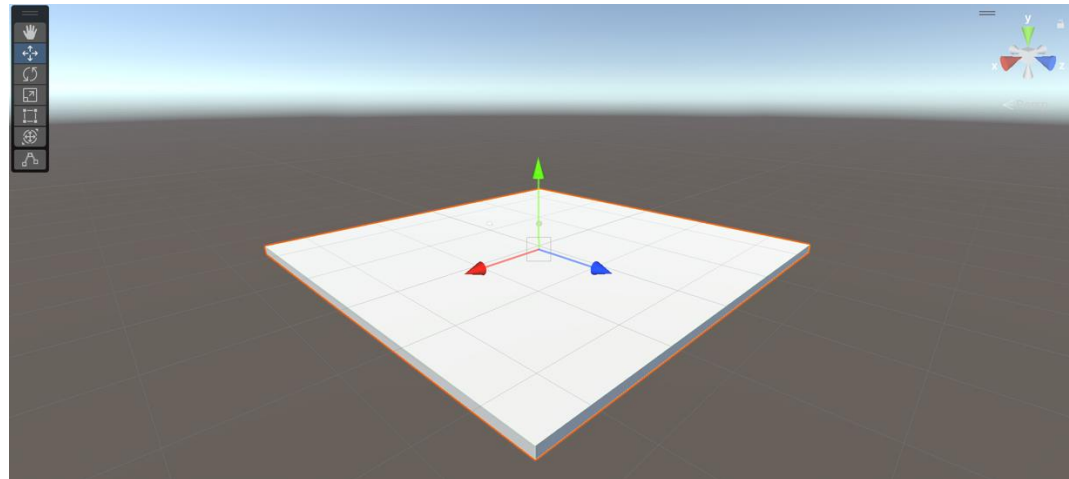
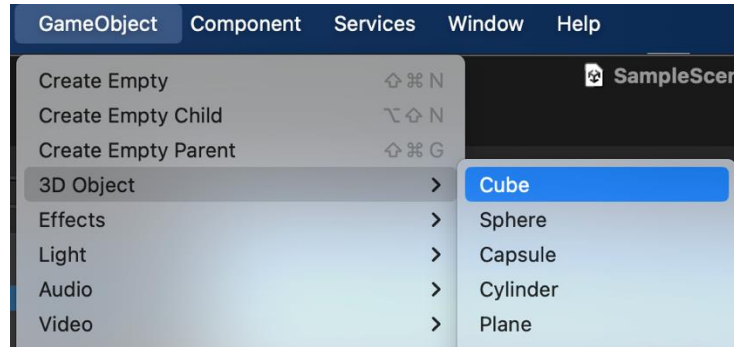
Starting from an **empty project**, Unity automatically includes **two essential objects** in the default scene:

- **Main Camera** – Defines the player's point of view, allowing us to see the scene.
- **Directional Light** – Simulates sunlight, ensuring objects are illuminated.

These components are fundamental for **rendering and visualizing** a scene in Unity.



Create a floor by using a cube

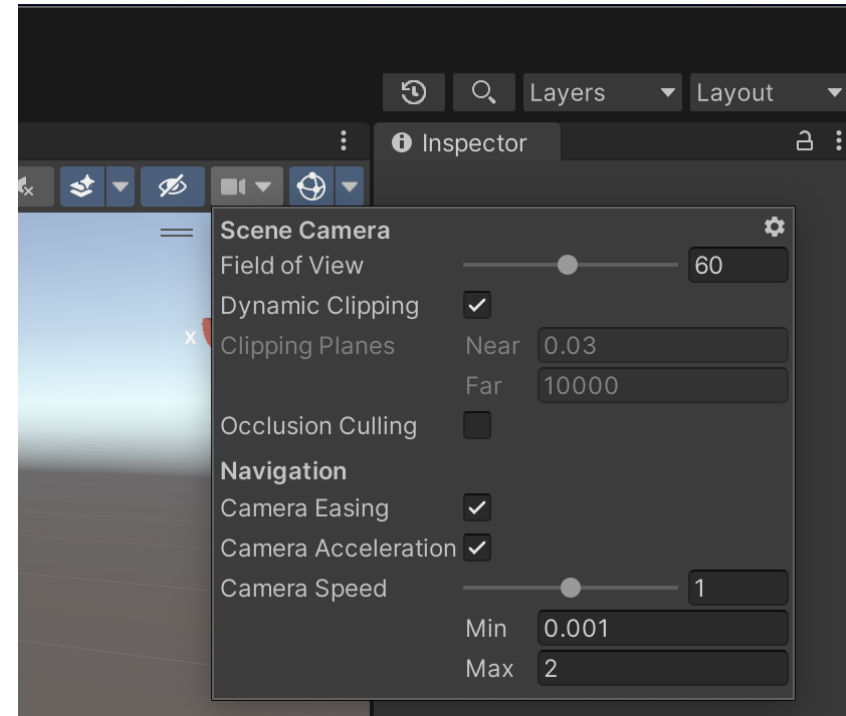


Object Name



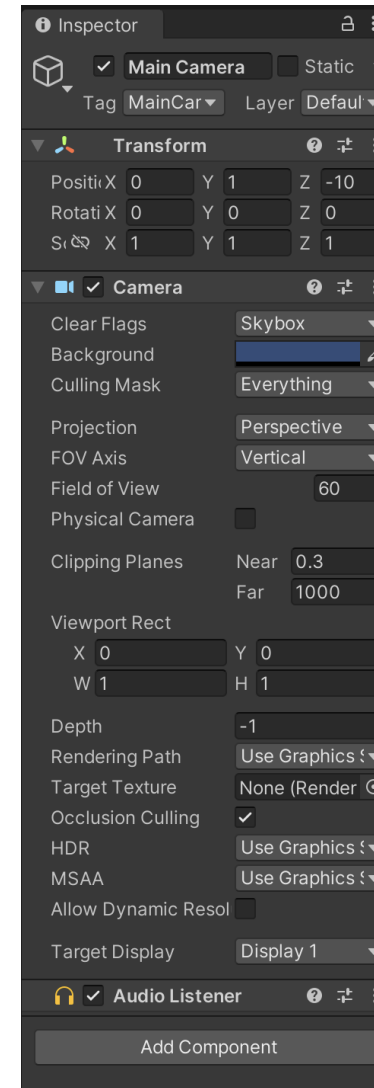
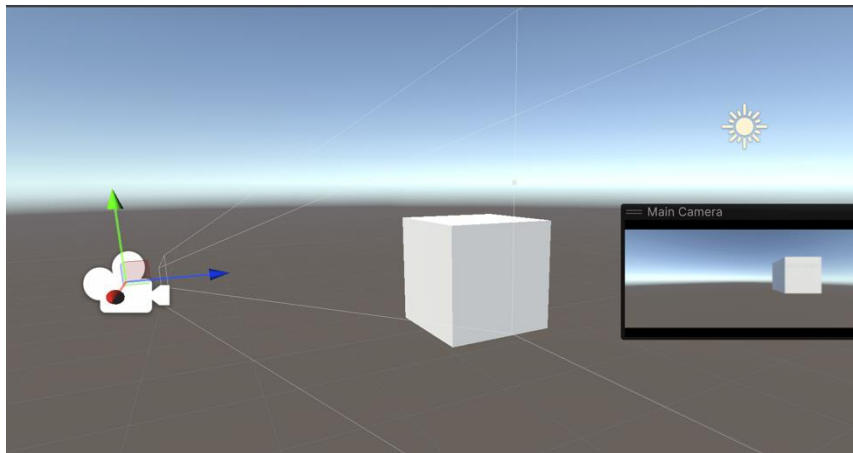
How do I move into the scene?

- The **scene view** has its own default camera.
 - It does not appear in the Project tab
 - It can be customized by the inspector
 - It has no visible gizmo
 - It has no visible clipping
 - It has nothing but it exists!!!



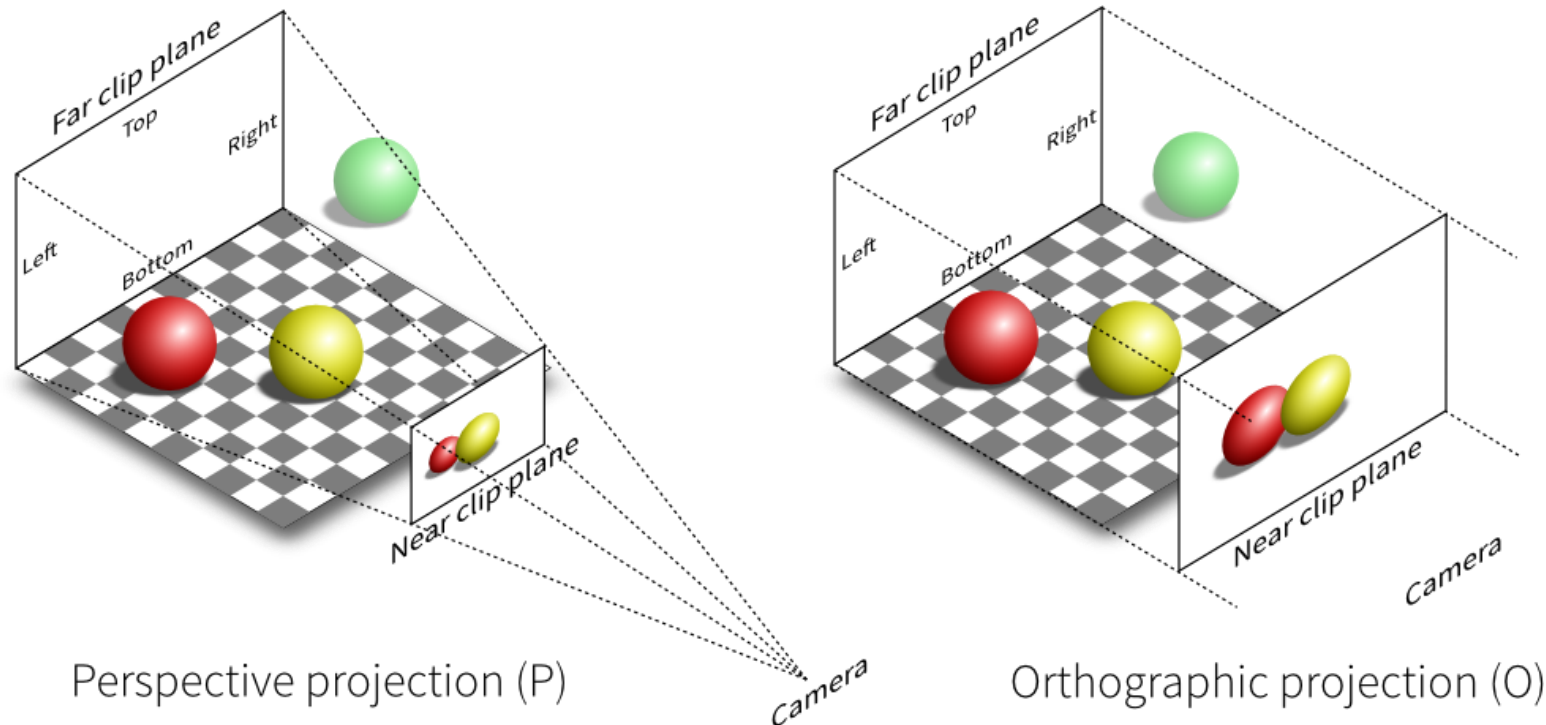
Main camera

- The **scene** has its own default **Main camera**.
 - It does not appear in the Project tab
 - It can be customized by the inspector
 - It has a preview
 - It has visible clipping



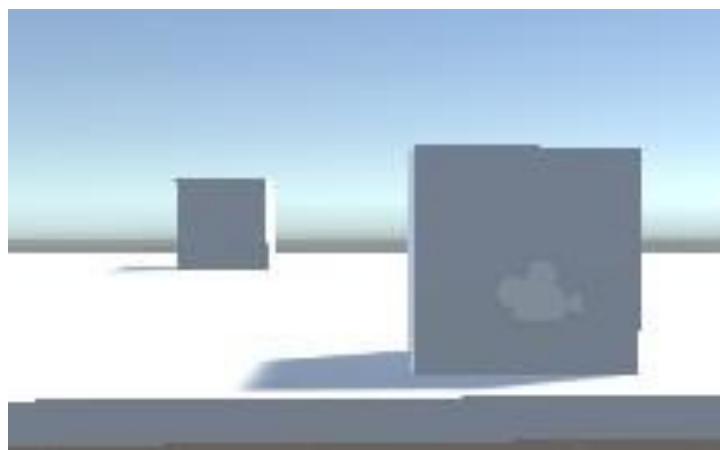
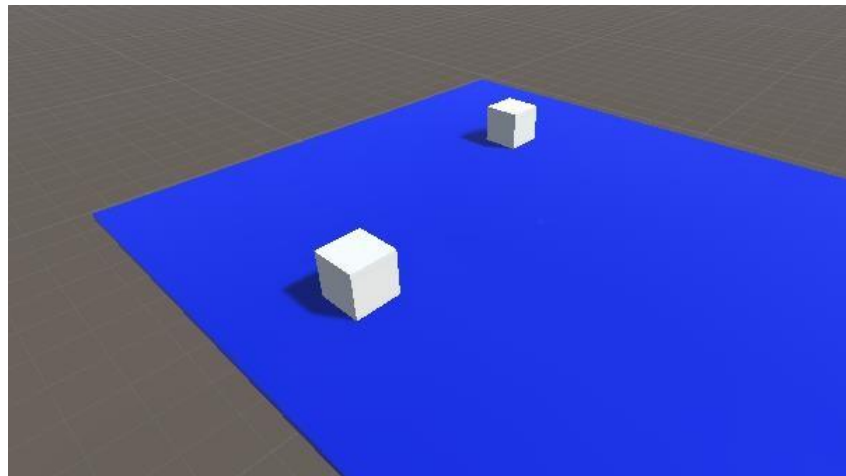
The scene camera views

- In the perspective view objects which are far away are smaller than those nearby.
- In the orthographic view, all objects appear at the same scale.

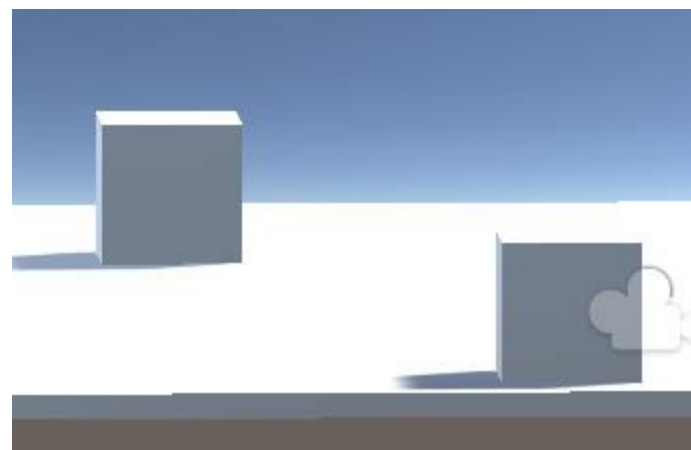


Perspective/Orthographic Projection

Top angled view

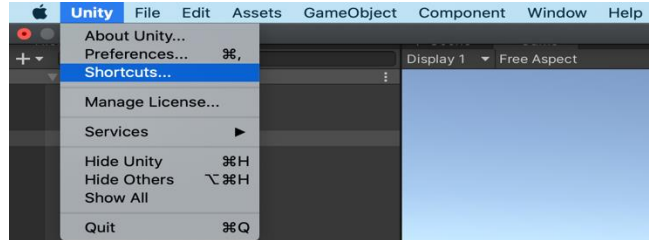


Perspective view

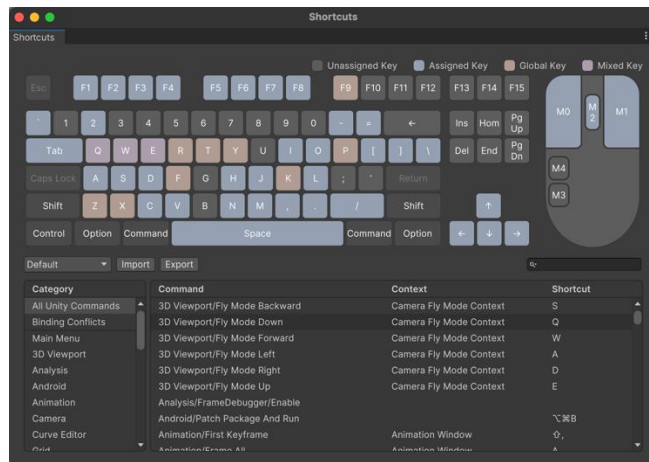


Ortho view

Shortcuts for scene control



Shortcuts may be customized via Edit/Shortcuts menu.



A) Profile drop-down menu: Allows you to manage, and switch between, different shortcut profiles.

B) Key map: Shows you which keys are assigned to commands, and the commands they are assigned to.

C) Category list: Commands are sorted into categories, which appear in the **Category** list. When you select a category, its commands appear in the **Command** list on the right.

D) Command and Shortcut list: Lists all commands in the selected Category and their assigned shortcuts.

<https://docs.unity3d.com/Manual/UnityHotkeys.html>

Control the Scene View via mouse and keyboard

- Mouse Left button
 - Object(s) selector (also Box selection)
- Mouse Right button
 - Rotate camera along its axes
- Mouse third button
 - Translate camera
- Alt (option ⌘) + left button: rotate along sceneview center
- Alt (option ⌘) + right button: zoom in/out
- Ctrl (command ⌘) + D: duplicate object
 - Ctrl (command ⌘) + C: copy object
 - Ctrl (command ⌘) + V: paste object

Units in Unity

- What does moving **K units** in Unity mean?
- Example: “How much translating 3 unit along X actually moves the object?”
- Answer: It translate of 3 units!!
- Unity has not customizable measurement units (e.g., meter, centimetre, inch and so on). What 1 units stands for is up to the developer.
 - It could be 1 meter or 1 centimetre or 1 inch.
 - Once measurement system chosen, it is important to be consistent all over the development of the application.

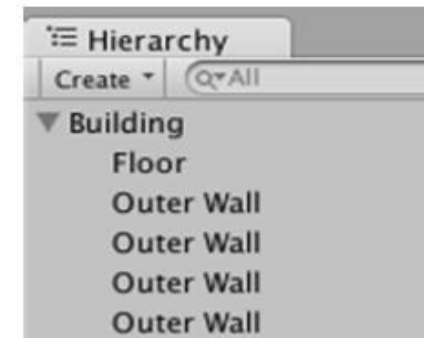
Attach object to other objects

- When developing a **3D scene**, it is useful to link related objects together to form a structured **group** in the **Hierarchy**.

How is This Done?

- Unity uses a **Parent-Child Relationship** to create hierarchical structures.
- **The parent object** (could also be an **empty GameObject**) acts as the main reference.
- **All other objects become children**, inheriting transformations like **position, rotation, and scale** from the parent.

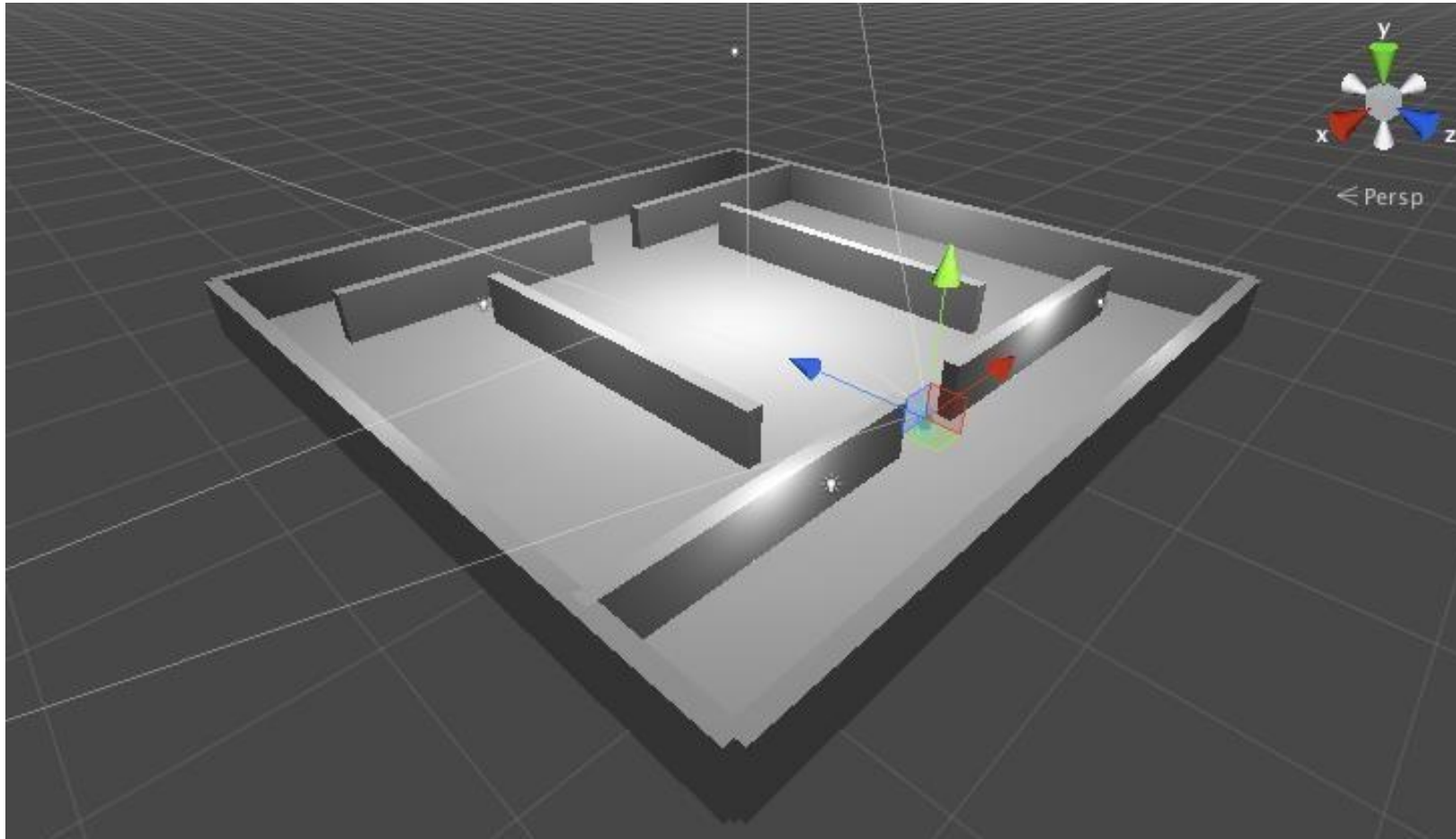
Example: A **building** as the parent, with floor and walls as child objects.



Parent/Child Relationship

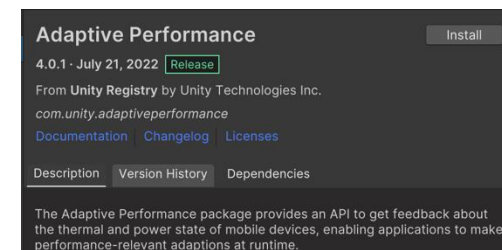
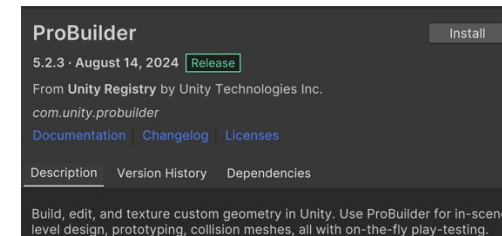
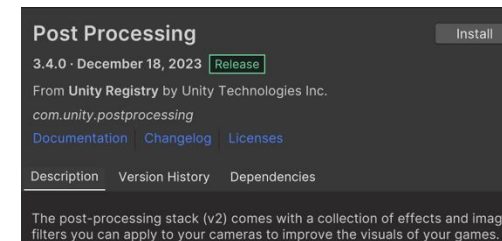
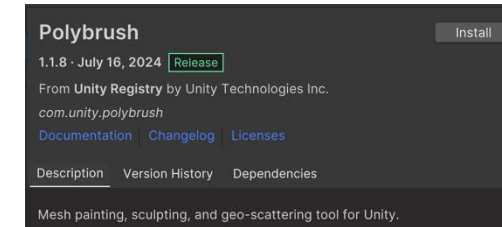
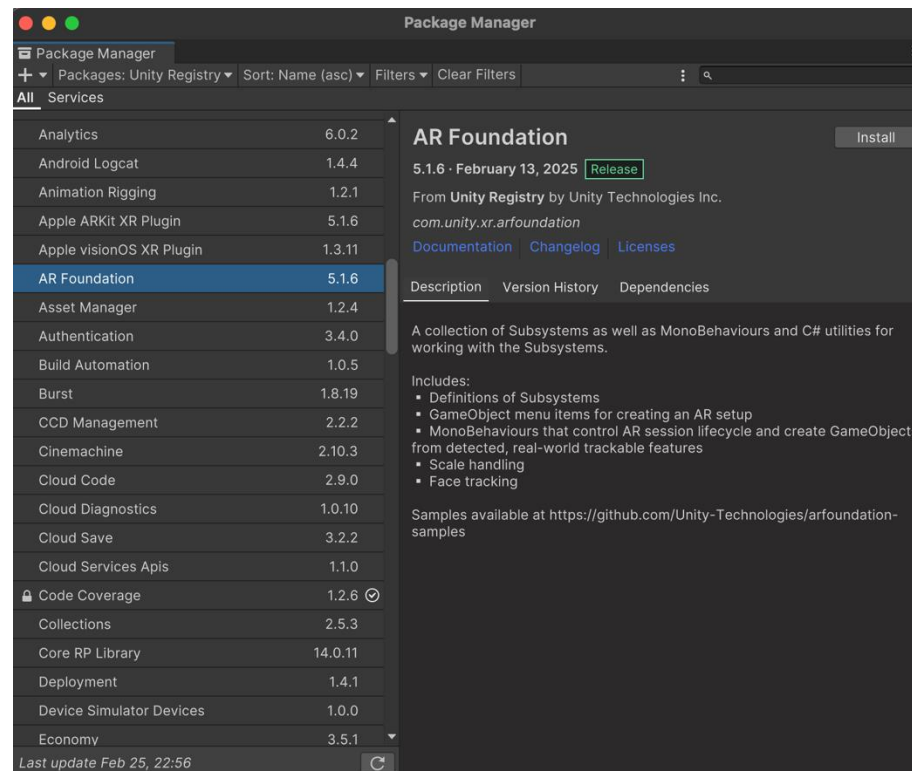
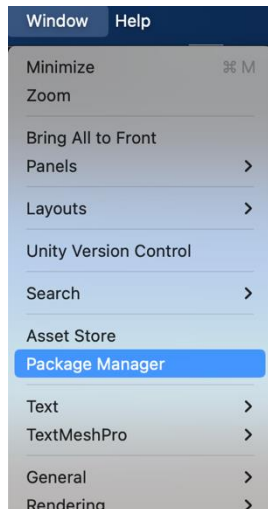
- A transformation applied to the parent affects all children.
- A transformation applied to the child has NO effect on the parent.
- Typical use case: camera attached to the character
 - The character is the parent: when character moves/rotates, the camera moves/rotates accordingly
 - The camera is the children: it can move/rotate even when the character is in idle mode.

Let's complete the scene!



Package Manager

- Menu Window - > Package Manager



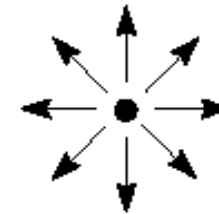
Light and Colour Space



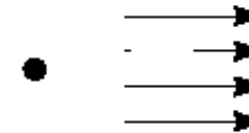
Lights

- In Unity, you can create different types of light sources to illuminate your scene realistically.
- The main types of light are:
 - **Directional**
 - **Point**
 - **Spot**
 - **Area** (Rectangle)
 - **Tube** (HDRP, Emits light in all directions equally along a line in space.)

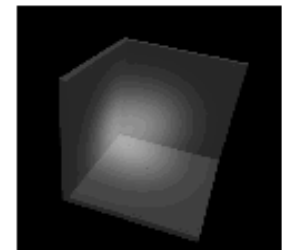
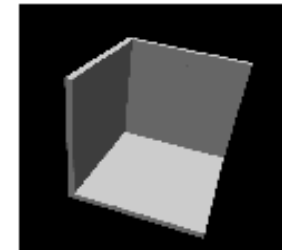
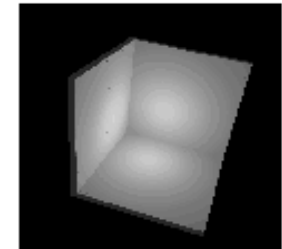
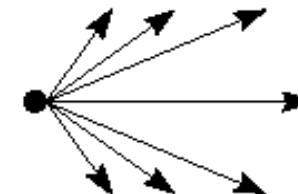
Point Light



Directional Light



Spot Light



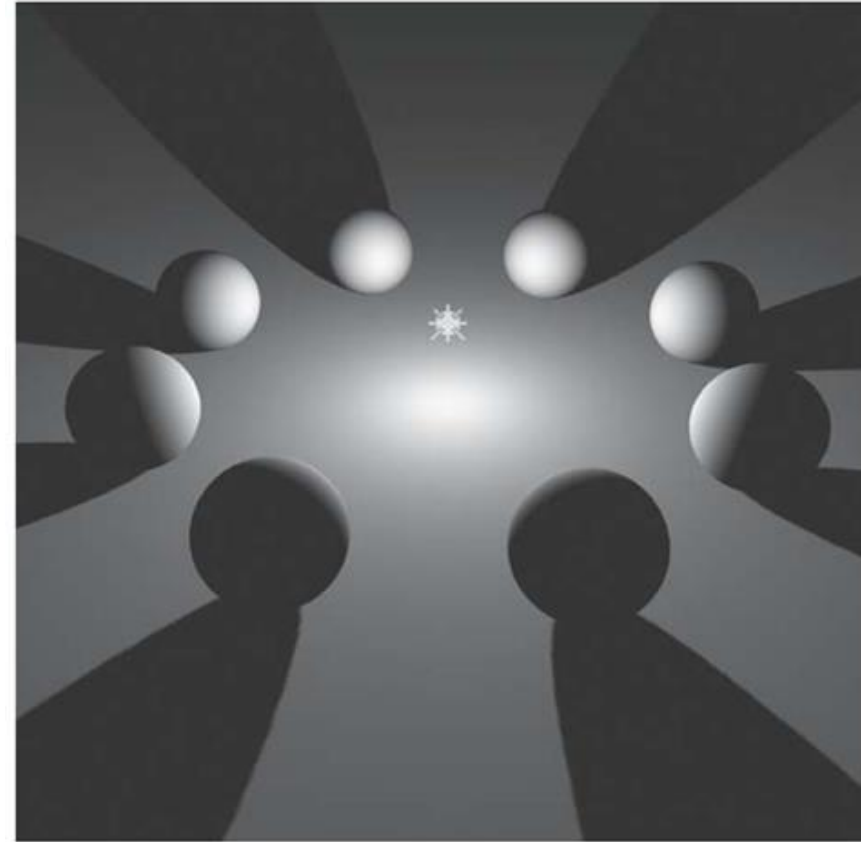
Directional Light

- A **Directional Light** is a light source where all rays are **parallel** and illuminate objects **evenly**.
- Lighting everything in the scene the same way.
- This is like the sun in the real world.
- The **position** of a Directional Light **does not matter**; only its **direction** affects the scene.



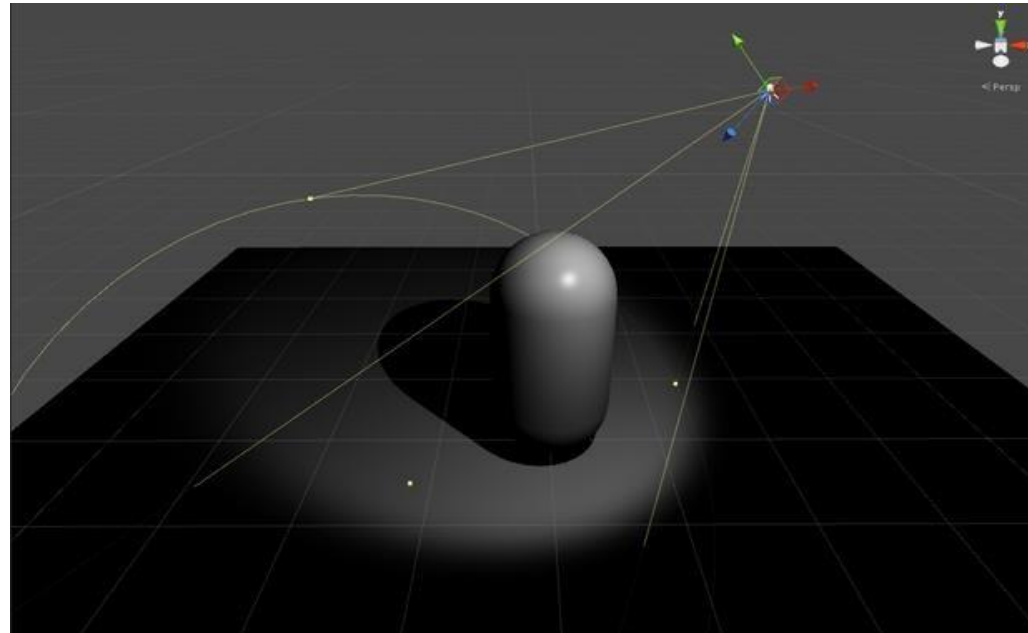
Point Light

- A **Point Light** is a light source where all rays originate from a **single point** and spread **in every direction**, similar to a **lightbulb** in the real world.
- **Emits Light in All Directions:** light spreads evenly around the source.
- **Brightness Decreases with Distance:** the closer an object is to the light, the brighter it appears.
- **Useful for Localized Lighting:** ideal for lamps, torches, and glowing objects.



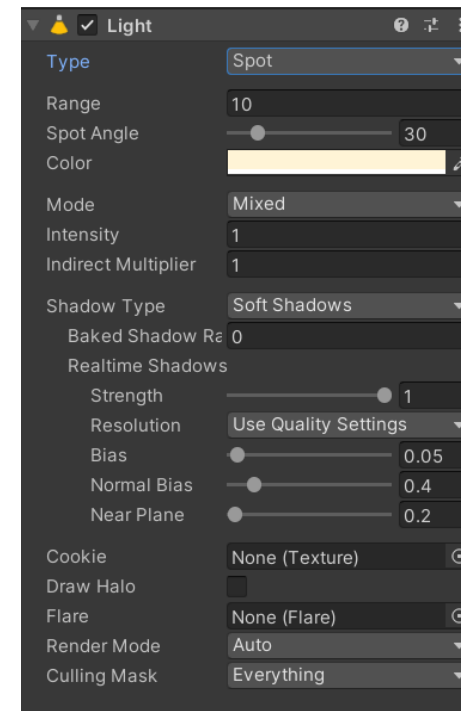
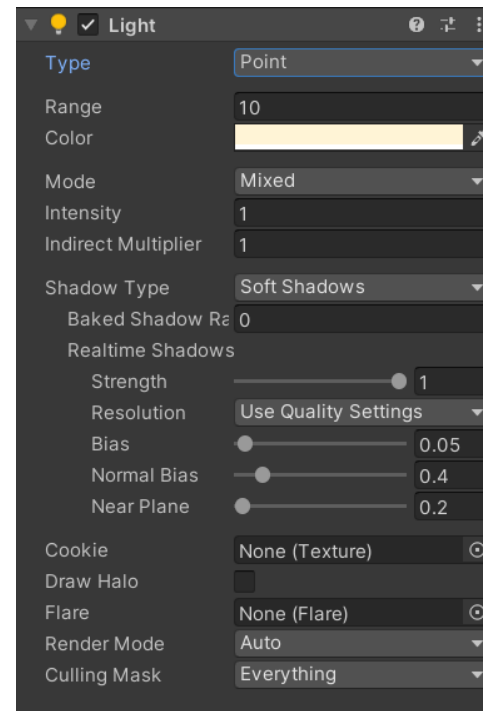
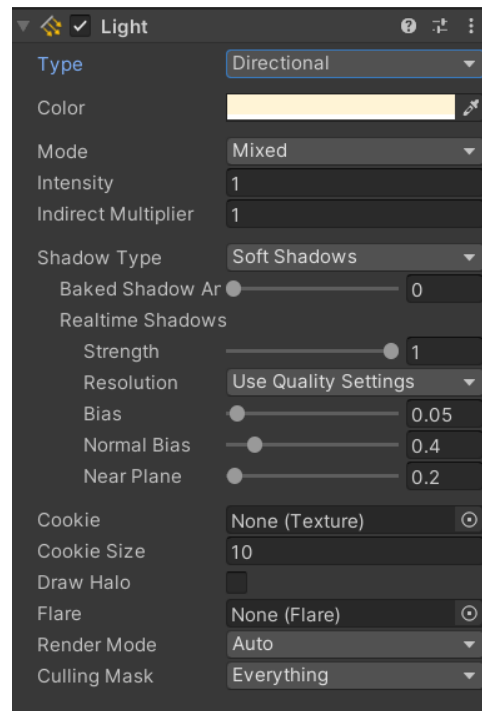
Spot Light

- A **Spot Light** is a light source where all rays come from a **single point** but are limited to a **cone-shaped beam**, similar to a **flashlight** or a **stage spotlight**.
- Only project out in a limited cone.
- Used for **flashlights**, **car headlights**, **stage lighting**, and **guiding players' attention to specific areas**.



Inspector for Lights

- Settings for colour, shadows, silhouette projection, brightness and bounce of the light and so on.
- Take into account that the inspector changes for point/spot/directional lights.

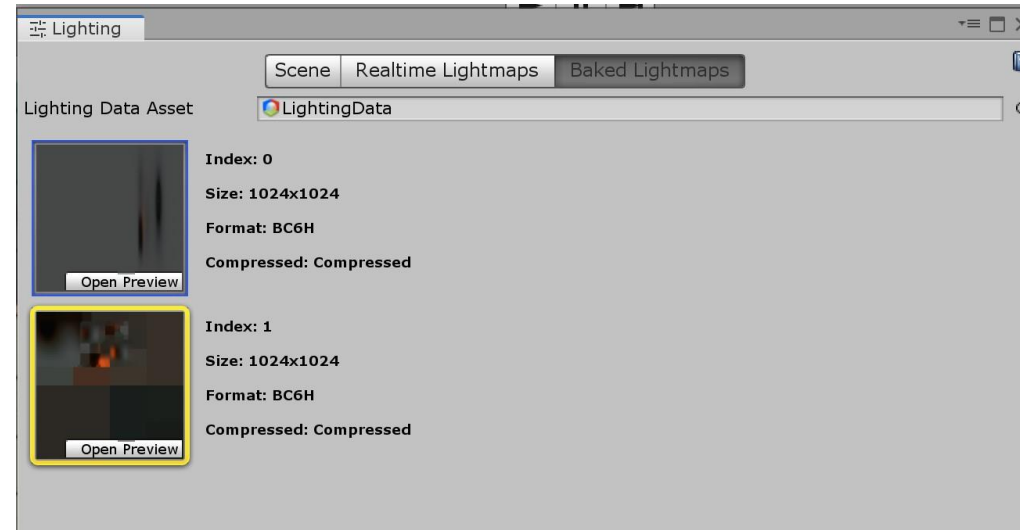
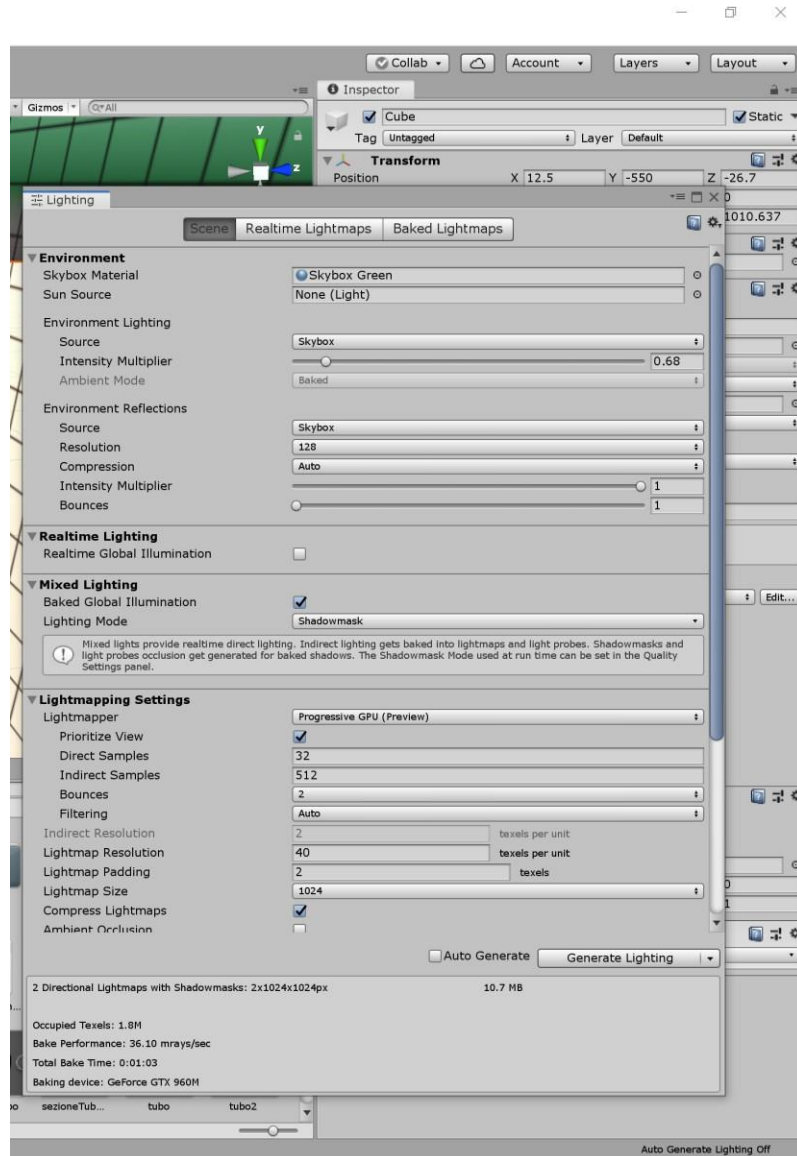


Baked or Realtime Lighting?

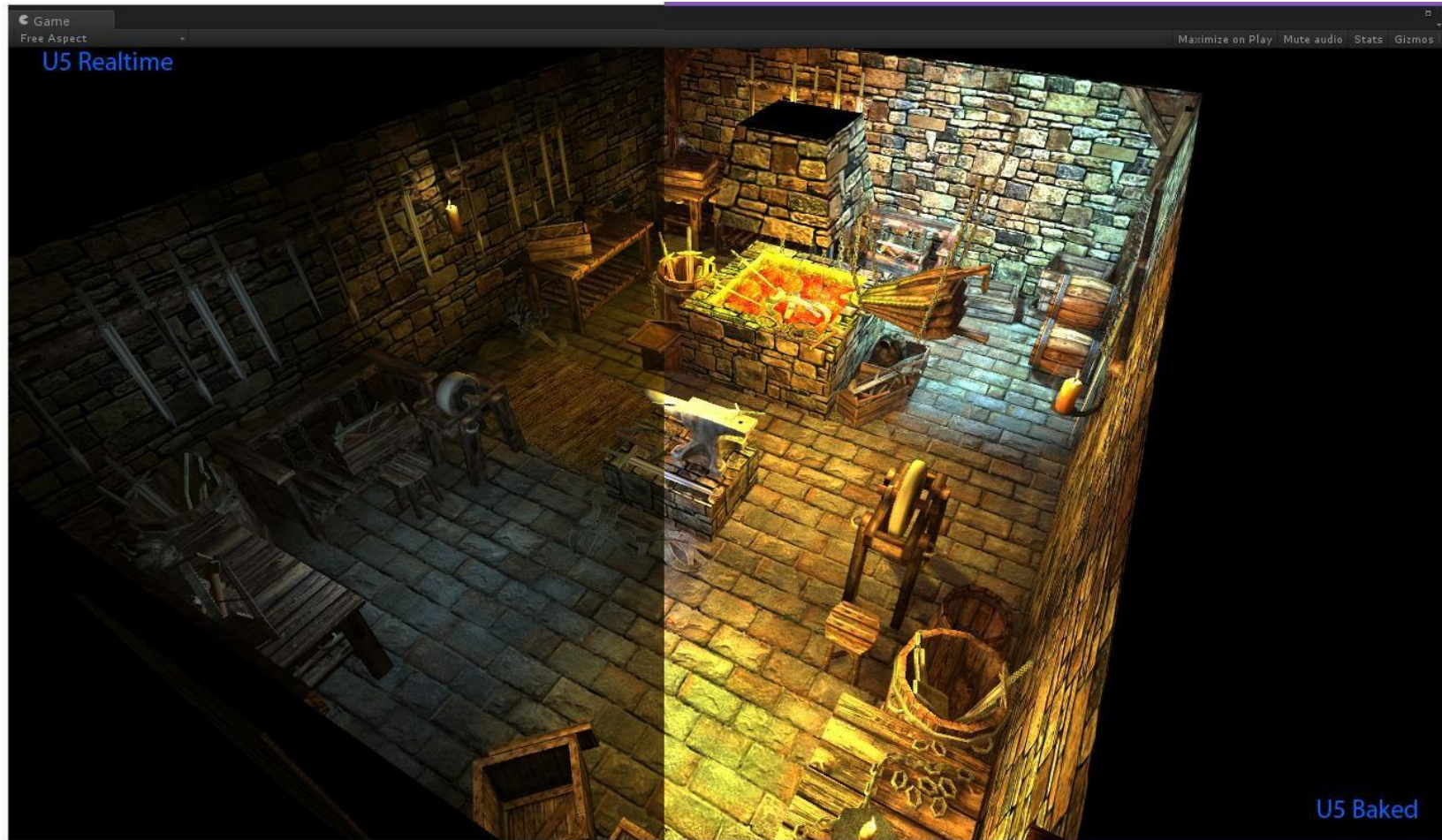
Lighting in Unity can be handled in **two main ways**, each with its own advantages and trade-offs.

- **Realtime lighting:** light effects are calculated at runtime.
 - Requires nearly no pre-computations for good looking results.
 - Dynamic objects receive the proper amount of light according to their position in the scene
 - High-quality real-time lighting and shadows require extra processing, potentially reducing performance – It Costs!!
- **Baked Lightmaps:** pre-computed lighting effects.
 - Affect the graphic performance much less than real-time lighting.
 - Light and shadows are calculated before runtime and stored in textures called lightmaps.
 - Lightmaps are like textures that contain light information instead of colour.
 - Only static objects can be included in the bake/precomputation
 - Open **Window** → **Rendering** → **Light** to access the Lightmapping settings

Baked or Realtime Lighting?



Baked or Realtime Lighting?



Mixed Lighting

- **Mixed lighting:** **combine** elements of both realtime and baked lighting.
 - Combines dynamic shadows with baked lighting from the same light source.
 - We can change properties of Mixed Lights at runtime: this updates their real-time lighting but not their baked lighting.
 - Illuminate both static and dynamic GameObjects, always provide direct lighting, and can optionally provide indirect lighting.
 - Mixed Lights always involve more runtime calculations than fully baked lighting and higher memory usage than fully real-time lighting.

An Example of Mixed Lights

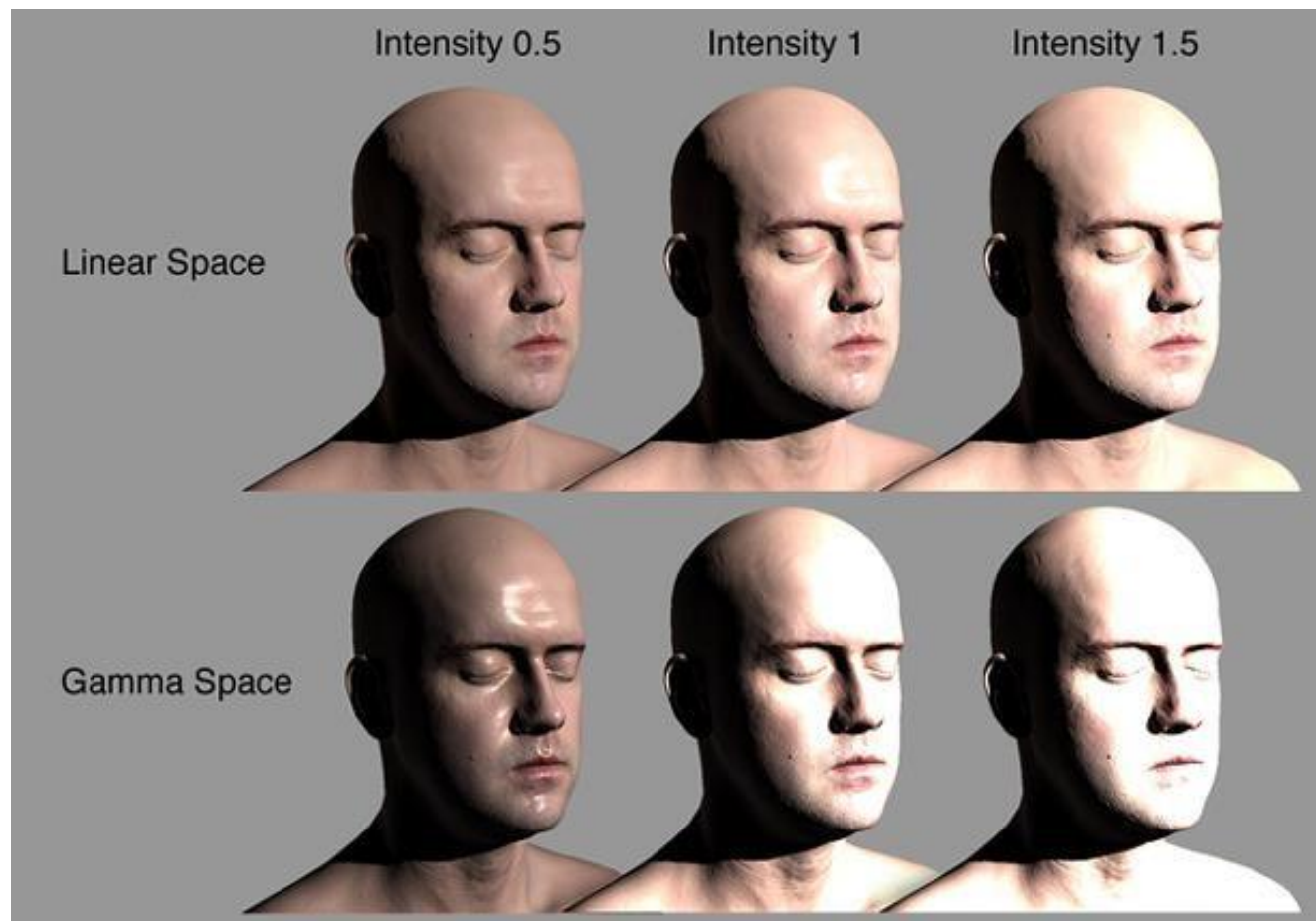


The colour space

- Colour Space determines **how Unity processes colours** when performing **lighting calculations and reading texture values**. Choosing the right colour space affects the **accuracy of lighting and shading** in your project.
- **Linear Colour Space** (*Recommended for Realistic Rendering*):
 - Preserves colour accuracy and realistic lighting.
 - Light intensity decreases/increases linearly with distance.
 - Best suited for physically-based rendering (PBR) and high-quality graphics.
 - Requires proper hardware support (not available on most mobile platforms).
 - How to enable: *Edit* → *Project Settings* → *Player*
- **Gamma Colour Space** (*Less Precise, but Faster*):
 - Less computationally expensive (better for performance).
 - Compatible with a wider range of devices, including mobile.
 - Does not preserve colour appearance correctly.
 - Lighting calculations are less accurate, leading to unrealistic results.

.25	+	.25	=	.50	Linear
.53	+	.53	=?	.73	Non-Linear

Linear vs Gamma Colour Space



Why Gamma Colour Space?

1. Real scenes often have non-linear response to light intensity
2. Human eyes are more sensible to darker shades than lighter shades
3. Linear shaders sample textures without Gamma (midtone) compensation. This helps to ensure that colour values remain consistent throughout their journey through the render pipeline

Materials, Shaders and Textures



Definitions

In Unity, **Textures, Materials, and Shaders** work together to define how objects **look and react to light** in the scene:

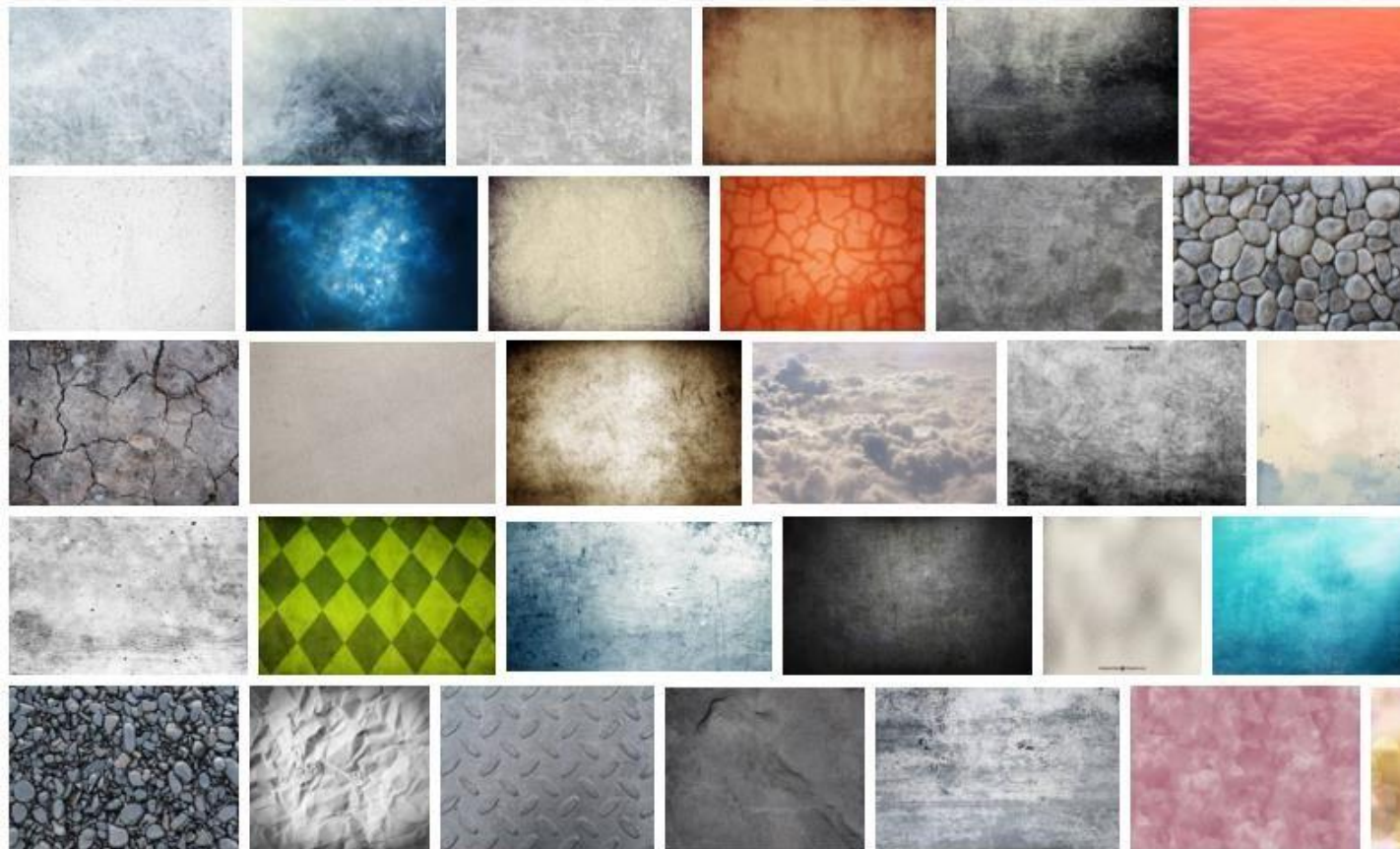
- **Materials** define how a surface should be **rendered**, including references to textures used, tiling information, colour tints and more. The available options for a material depend on which shader the material is using.
- **Shaders** are small scripts that determine how pixels appear on a surface. They use **mathematical calculations** to compute **lighting, shadows, and effects** based on the Material's settings. Different shaders create different effects.
- **Textures are bitmap images**. A Material may contain references to textures, so that the Material's shader can use the textures while calculating the surface colour of an object. In addition to basic colour (albedo) of an object's surface, textures can represent many other aspects of a material's surface such as its reflectivity or roughness.

About Textures



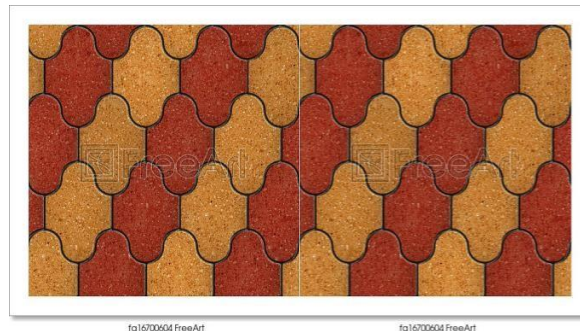
Textures

A texture is an image.



Textures Properties

- To achieve the best results in a **game environment**, a texture should have the following properties:
 - **Be tilable**: cover wide surface without making connections visible.
 - **Controlled light**: the illumination of the texture should support the illumination in the scene and not conflict with it.
 - **Resolution/Realism compromise**: High-resolution textures look great but **increase memory usage and rendering costs**. Finding a balance is key!

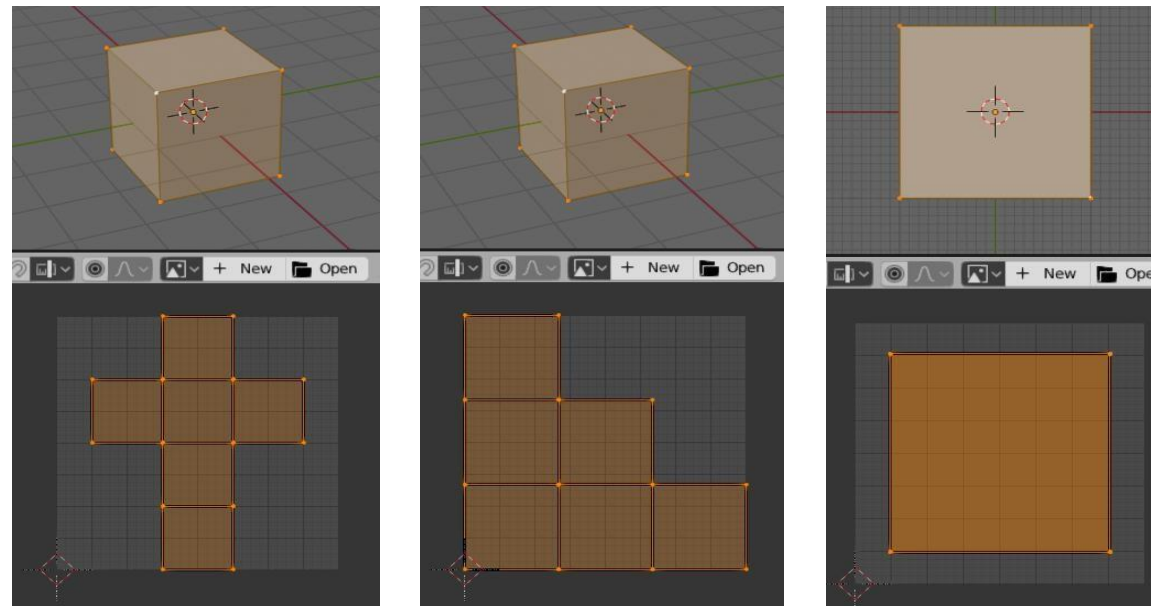


Textures should have dimensions that are powers of two (e.g., 256x256, 512x512, 1024x1024). This optimizes memory usage and rendering performance.

UV Mapping



- UV Mapping is the process of projecting a texture map onto a 3D object.
- The letters "U" and "V" denote the axes of the 2D texture.

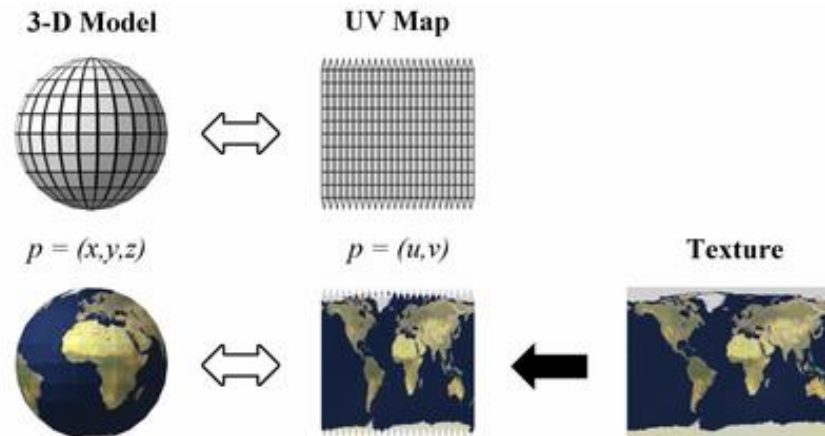


UV Mapping



- Assuming that the sphere's poles are aligned with the Y axis, UV coordinates in the range $[0,1]$ can then be calculated as follows:

$$u = 0,5 - \frac{\arctan 2(d_z, d_x)}{2\pi} = \frac{\pi - \arctan 2(d_z, d_x)}{2\pi}$$
$$v = 0,5 - 2 \cdot \frac{\arcsen(d_y)}{2\pi} = \frac{\pi - 2\arcsen(d_y)}{2\pi}$$



UV Mapping (more complex example)

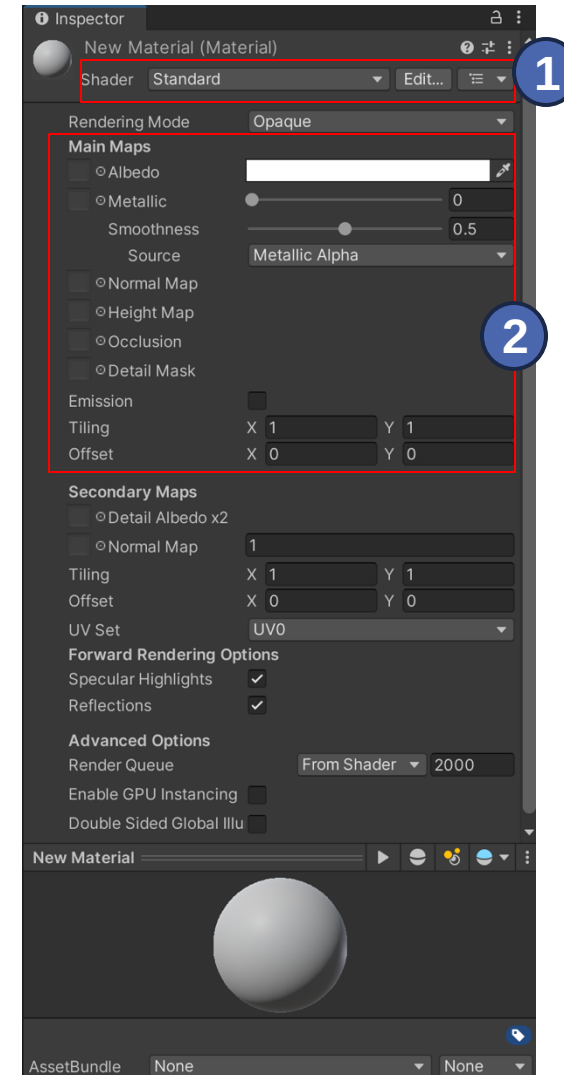


Materials and Shaders



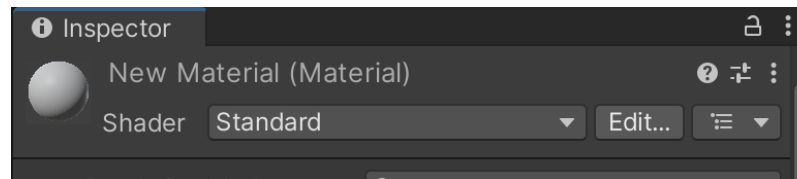
Creating new Materials

- To create a new Material, use **Assets->Create->Material** from the main menu or the **Project View** context menu
- By default, new materials are assigned the **Standard Shader** ① with all map properties empty ②
- Once created, just **drag** and **drop** the material on the desired object in the scene view



What is a Shader?

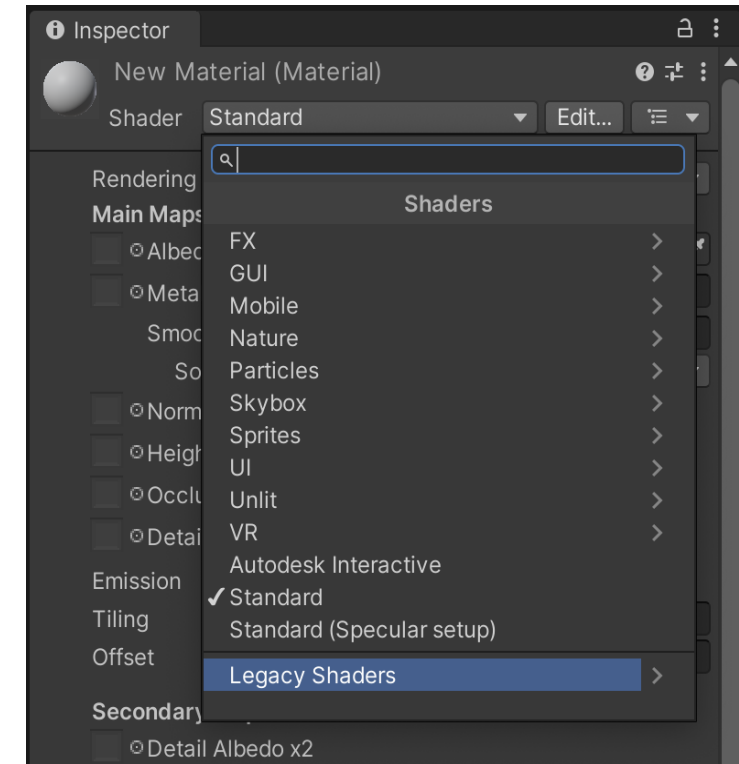
- A **Shader** is a script which contains mathematical calculations and algorithms for how the pixels on the surface of a model should look.
- The standard shader performs complex and realistic lighting calculations.
- Other shaders may use simpler or different calculations to show different results.
- Within any given Shader are a number of properties which can be given values by a Material using that shader.



Built-in Shaders

In addition to the **Standard Shader**, Unity provides a variety of **built-in shaders** designed for specific rendering needs:

- **FX:** lighting and glass effects.
- **GUI and UI:** for user graphic interface.
- **Mobile:** simplified high-performance shader for mobile devices.
- **Nature:** for trees and terrain.
- **Particles:** particle system effects.
- **Skybox:** for rendering background environments behind all geometry.
- **Sprites:** for use with the 2D sprite system.
- **Toon:** cartoon-style rendering.
- **Unlit:** for rendering that entirely bypasses all light & shadowing.
- **VR:** simple spatial mapping.
- **Standard and Standard (Specular setup)**
- **Legacy:** the large collection of older shaders which were superseded by the Standard Shader



Standard Shader

- The Unity **Standard Shader** is a **versatile, built-in** shader designed to provide **realistic rendering** for a wide variety of materials.
- Supports **multiple material types** such as stone, wood, glass, plastic and metal, and supports a wide range of shader types and combinations.
- Uses **Physically Based Shading** (PBS) to simulate real-world lighting interactions for greater realism.
- Supports **two workflows** for defining how materials interact with light: **Metallic** and **Specular** setups. Both methods determine how light reflects off a surface but differ in how specular reflections are controlled.

Standard shader: an example

A scene rendered using the standard shader on all models

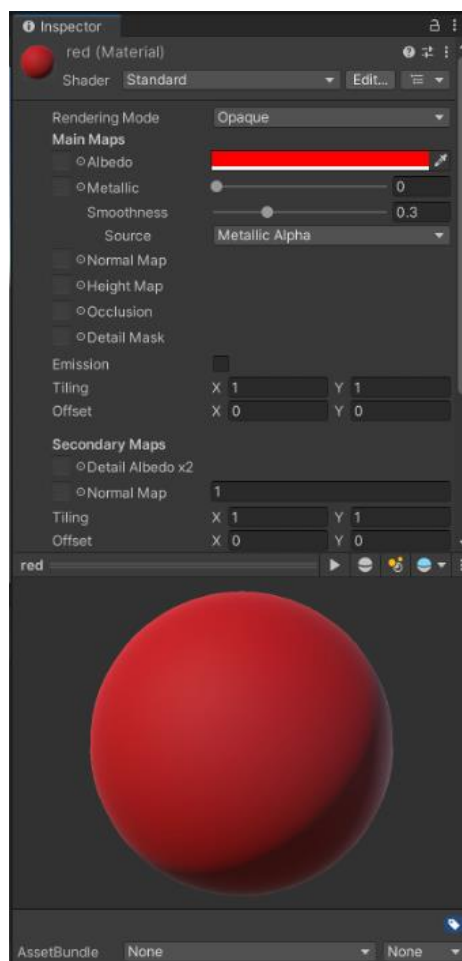


Standard VS Specular Setup

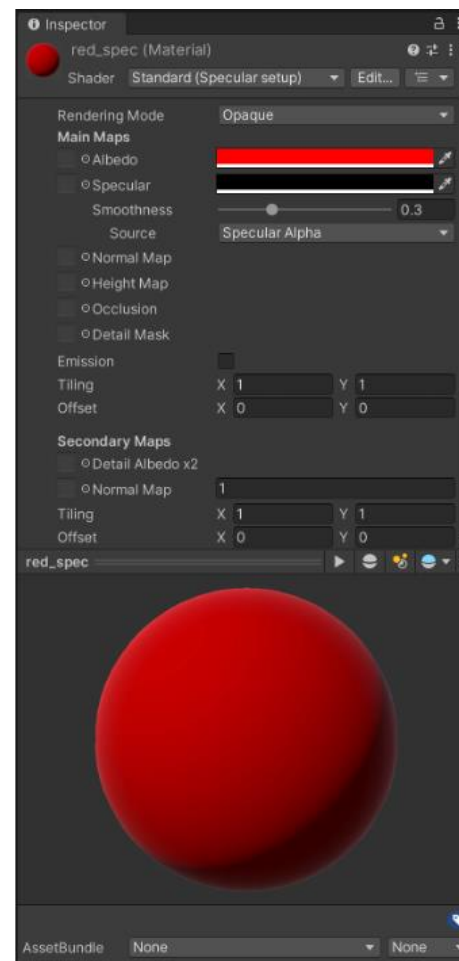
- **Standard:** uses a **Metallic value** to define if a material is **metal or non-metal**:
 - For **metallic** surfaces:
 - The **Albedo** defines the base colour.
 - **Metallic** texture controls reflectivity.
 - Most light is reflected as **specular reflections**.
 - For **non-metallic** surfaces:
 - Reflections take on the **colour of the light source** (not the object).
 - The surface reflects much **less light when viewed directly from the front**.
- **Standard (Specular setup):** provides direct control over **Specular reflections** using a **Specular Colour** value:
 - Specular reflections can have a **different colour** than the object's diffuse colour.
 - Allows **precise adjustments** to the **strength and colour** of reflections.

Metallic Vs Specular Setup: example1

Standard Metallic

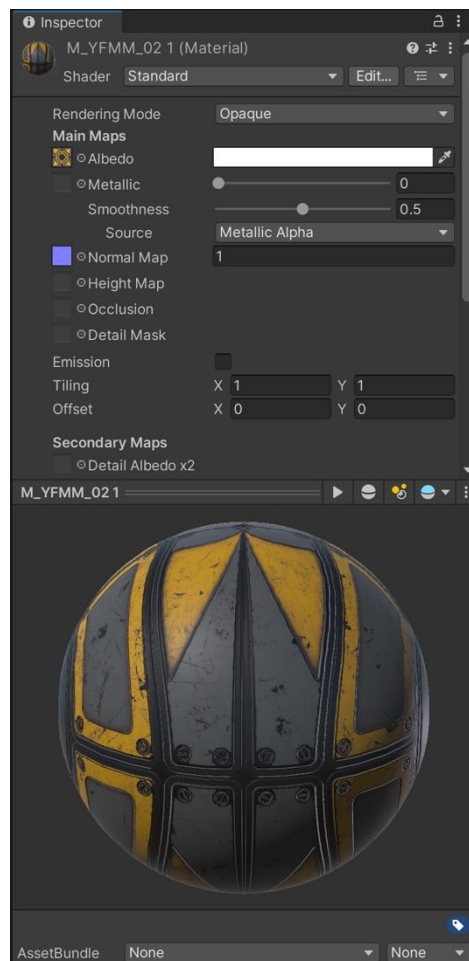


Standard
(Specular setup)

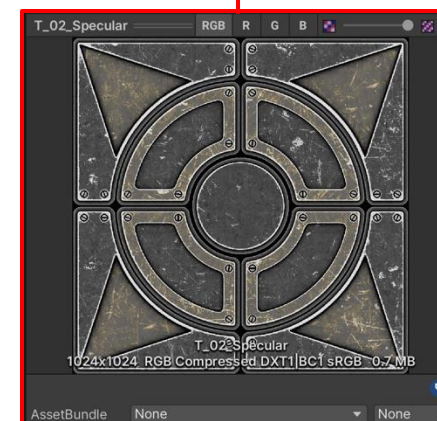
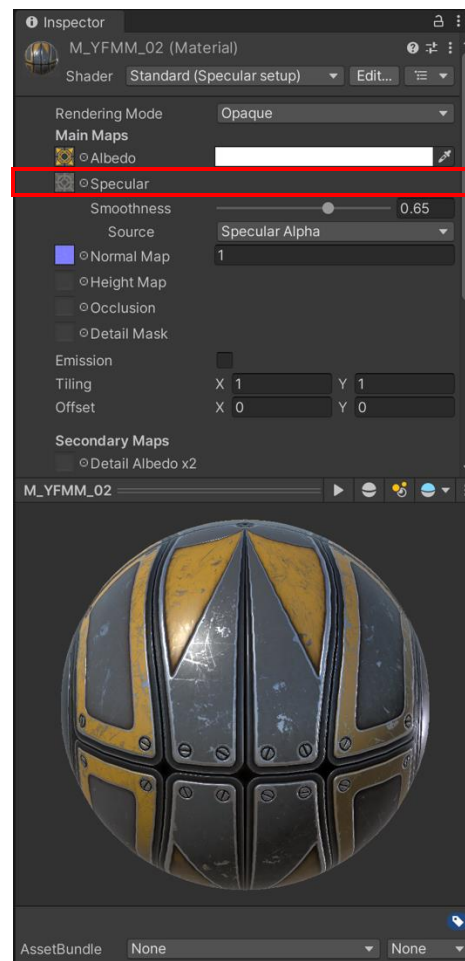


Metallic Vs Specular Setup: example2

Standard Metallic

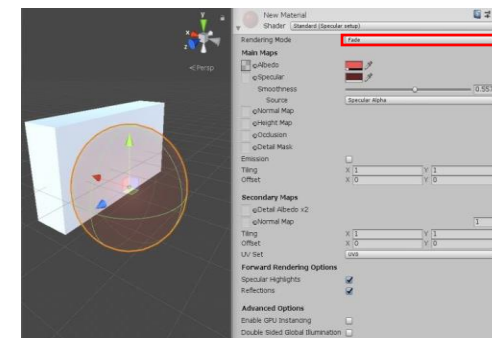
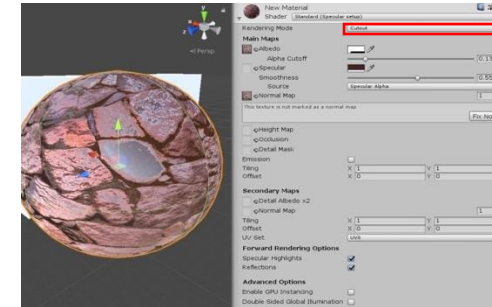
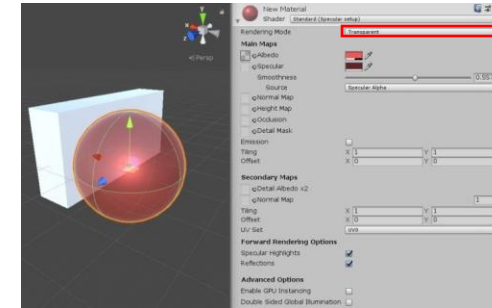


Standard
(Specular setup)



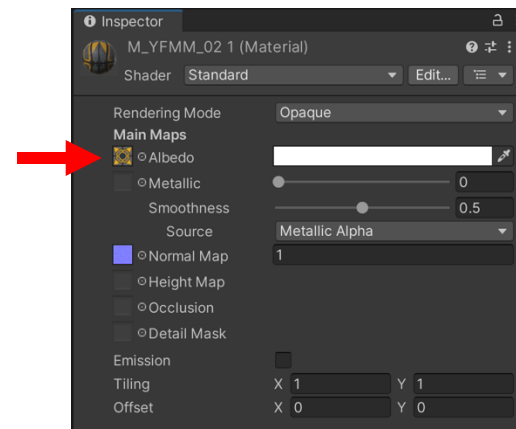
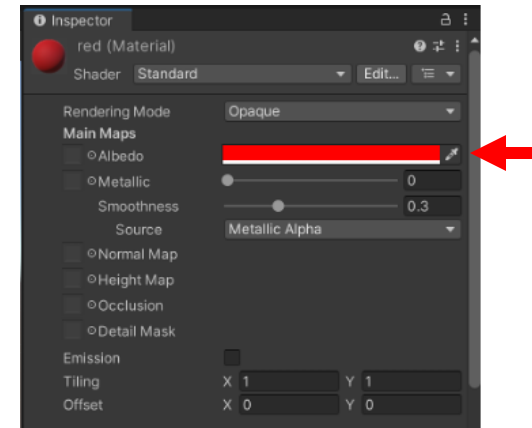
Material Attributes: Rendering Mode

- The **Rendering Mode** in Unity's **Standard Shader** controls how a material is **rendered and blended with the background**, affecting transparency and appearance.
- **Opaque (default)**: is suitable for normal solid objects with no transparency.
- **Cutout**: creates sharp-edged transparency with hard edges between the opaque and transparent areas.
 - This is useful when using transparency to create the shape of materials such as leaves, or cloth with holes and tatters.
- **Transparent**: suitable for rendering materials with smooth transparency like glass, clear plastic, or water.
- **Fade**: works like Transparent mode, but also removes reflections and specular highlights as the object fades out.
 - Ideal for animated fading effects, such as making objects gradually disappear.



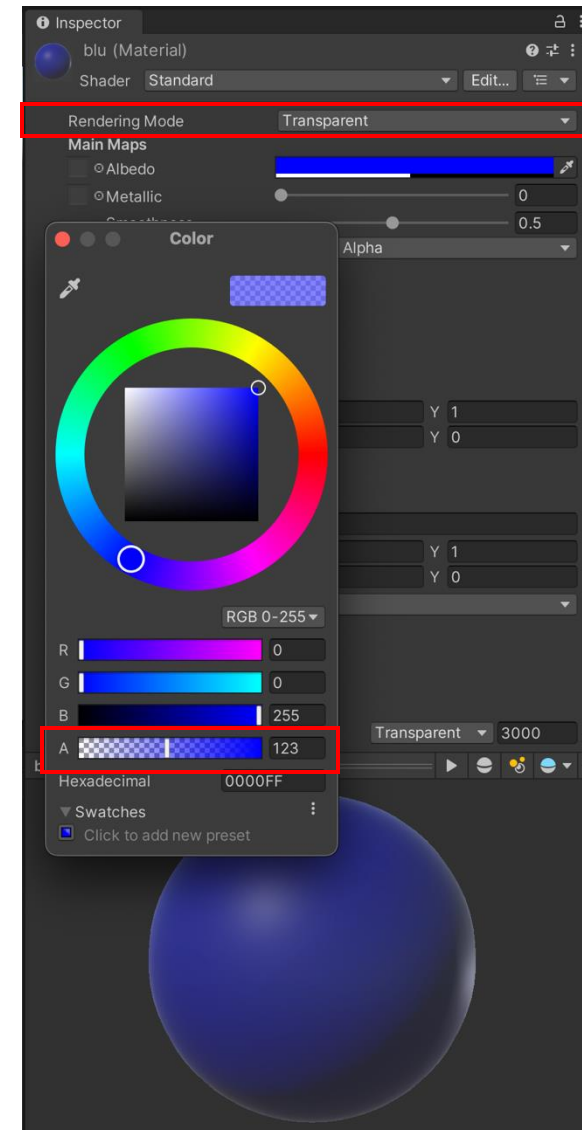
The Albedo Attribute

- The **Albedo** parameter defines the **base colour** of the surface.
- Instead of a single colour, it is common to use a **texture** to achieve a more **realistic appearance** with natural variations.
- The **Albedo texture** does not include lighting information; it represents the **intrinsic colour** of the material before any shading or reflections are applied.



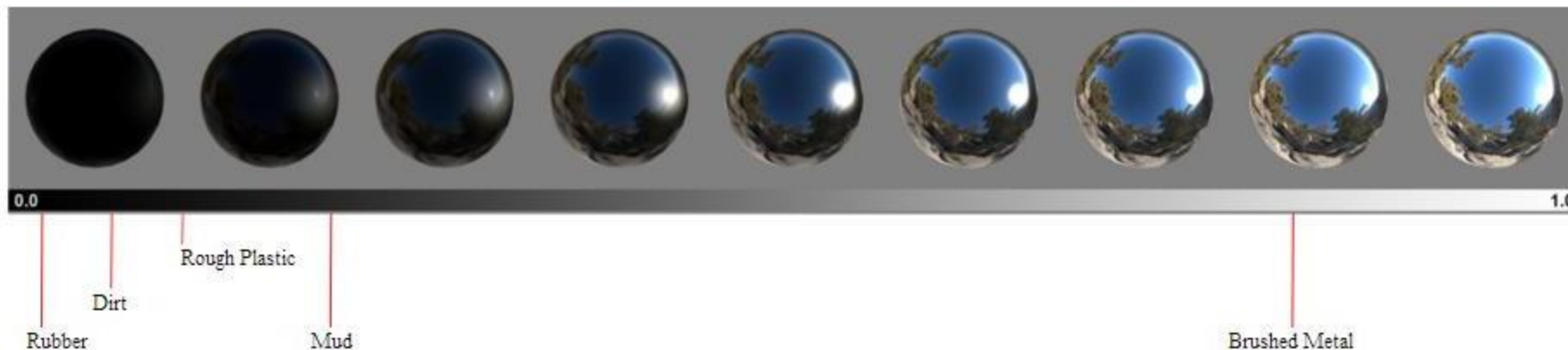
Albedo and transparency

- **Albedo** property can control **transparency**.
- The **alpha** value determines the material's **opacity**, allowing for **transparent** or **semi-transparent** effects.
- Transparency behavior depends on the **rendering mode** (Opaque, **Cutout**, **Fade**, or **Transparent**) set in the material settings.



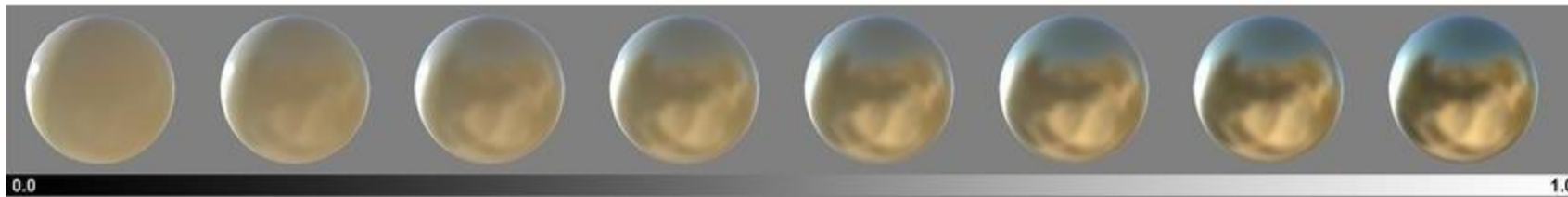
Smoothness (Specular Setup)

- The **Smoothness** parameter controls the **sharpness and clarity** of specular reflections.
- **Low Smoothness** → Reflections appear **blurred and diffuse**, even if they are strong.
- **High Smoothness** → Reflections become **crisper and more defined**, resembling polished surfaces.
- Higher smoothness values simulate materials like **glass or polished metal**, while lower values create surfaces like **matte plastic or rough wood**.



Smoothness (Metallic mode)

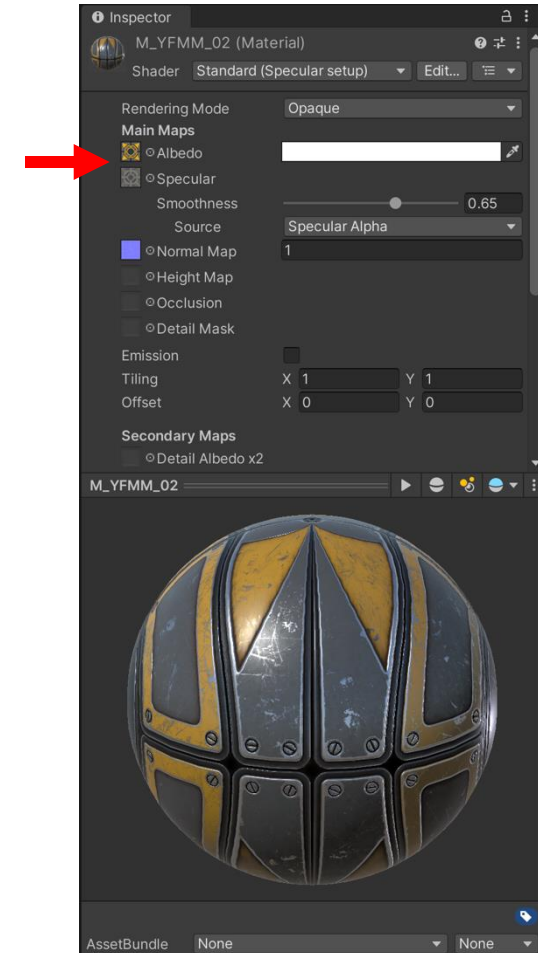
- In **Metallic Mode**, the **Smoothness** parameter controls the **strength and sharpness** of reflections.
- **Low Smoothness (≈ 0)** → Produces a **non-metallic appearance** with **diffuse, soft reflections** (e.g., rubber or matte surfaces).
- **High Smoothness (≈ 1)** → Creates **strong, sharp reflections**, simulating highly polished metals or glass.



Albedo and Specular/Metallic Texture



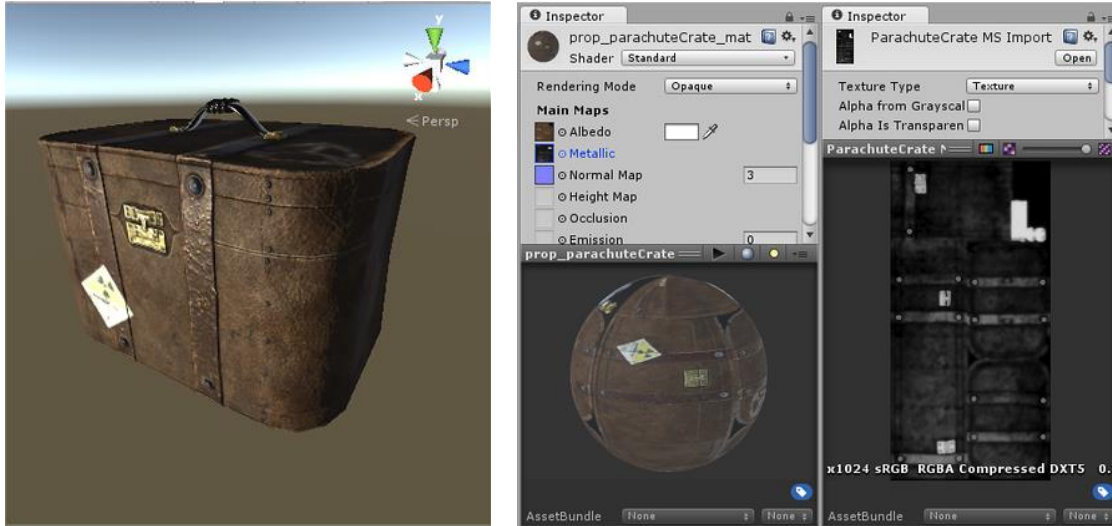
- Quite often, sliders are sufficient to get a high level of realism.
- However, when working with more complex models, textures can achieve nicer effects.
- **Albedo Textures:** control the base colour of the model surface
- **Specular/Metallic Textures:** control the reflective effect by using the grayscale intensity of the texture.



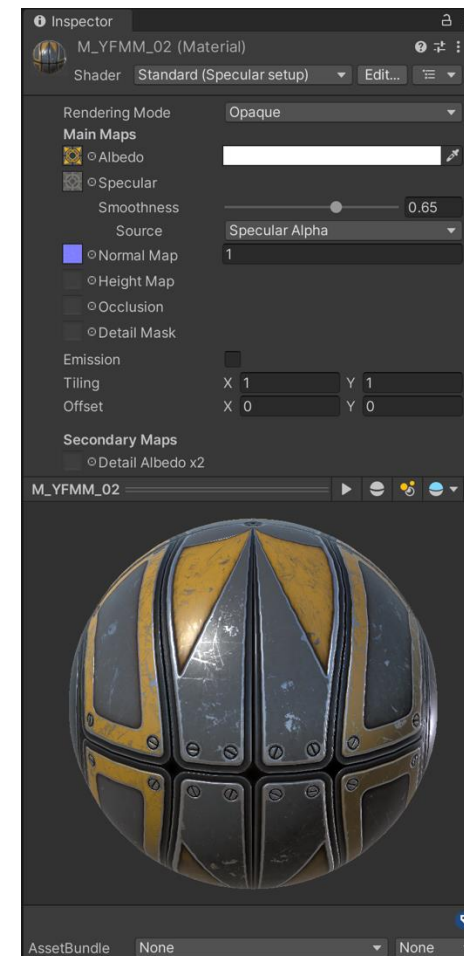
Specular and Metallic Texture



Metallic Map (Standard Shader)



Specular Map (Specular Setup)



Reflections and Skybox



About reflections

- The **materials' properties** in the **Inspector** control the **appearance** of a model, including how it reflects light.
- By default, **reflections** are based on the **Skybox**, which acts as the main environment source for reflective effects.

What is the Skybox?

- A **shader** that defines the **visual environment** surrounding the scene.
- Represents the **horizon** and **sky** in the scene, giving the illusion of depth and atmosphere.

6-sided Skybox

The **Six-Sided Skybox** is a simple and effective way to create a realistic environmental backdrop.

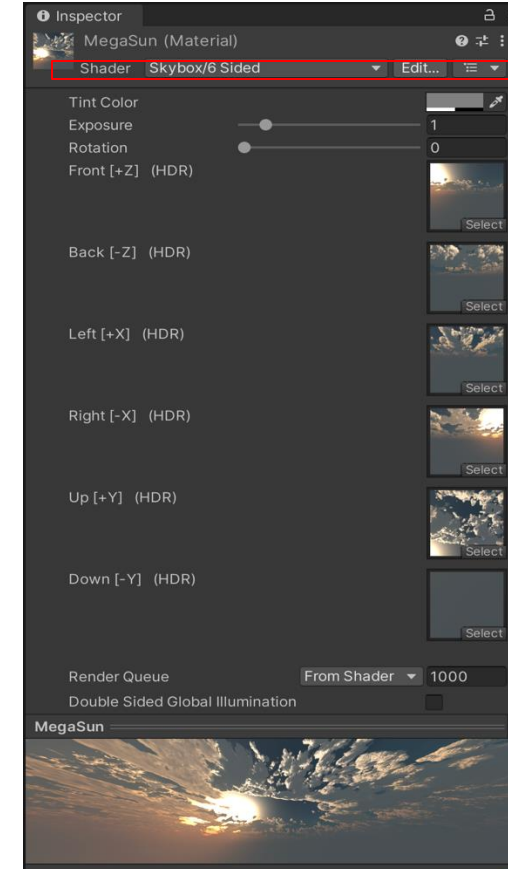
- **Steps to Set Up a Six-Sided Skybox:**

1. Create a New Material → Go to **Assets** → **Create** → **Material**.

2. Set the Shader → In the **Inspector**, change the shader to **Skybox/6 Sided**.

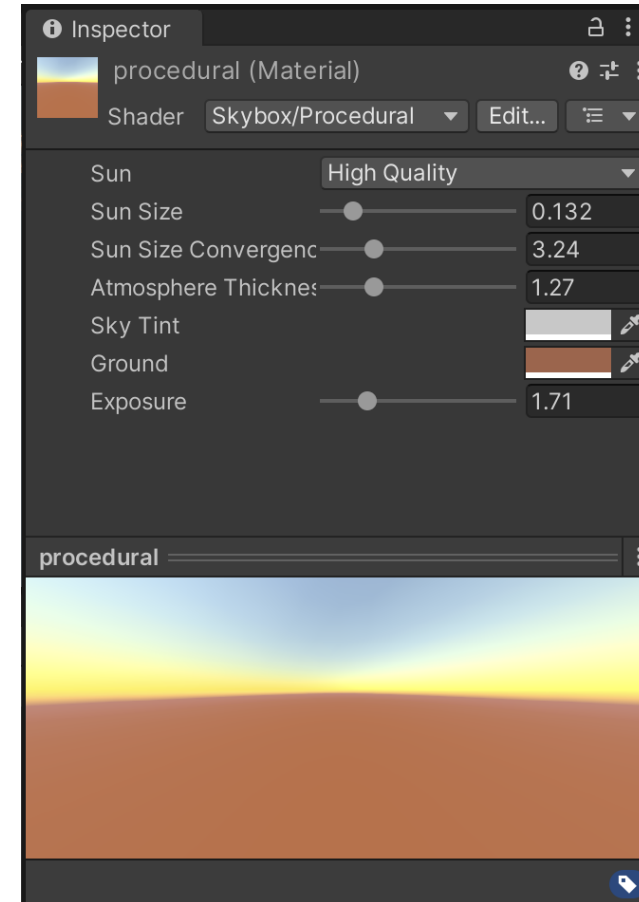
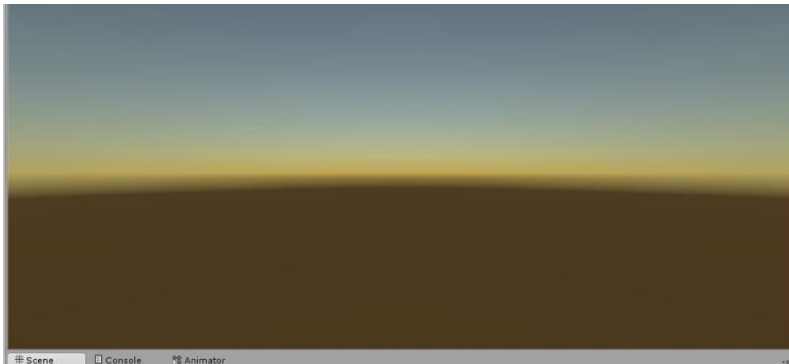
3. Assign Textures → Drag and drop six different textures (one for each direction) into the corresponding texture slots (**Front, Back, Left, Right, Up, Down**).

Provides a **seamless 360° environment** around the scene, enhancing reflections and realism.



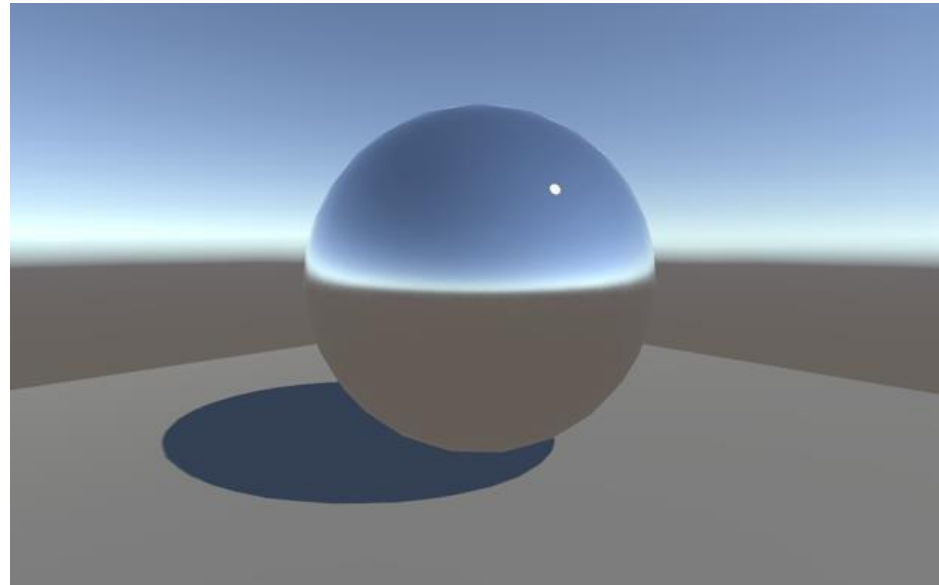
Procedural Skybox

- The **Procedural Skybox** dynamically generates the sky using **computed lighting effects** instead of static textures.
- The **sunlight effect** is calculated in real-time, providing a **natural and dynamic sky appearance**.
- The **inclination of the Directional Light** (representing the sun) **automatically affects the atmosphere's rendering**, simulating realistic day-night transitions and lighting changes.



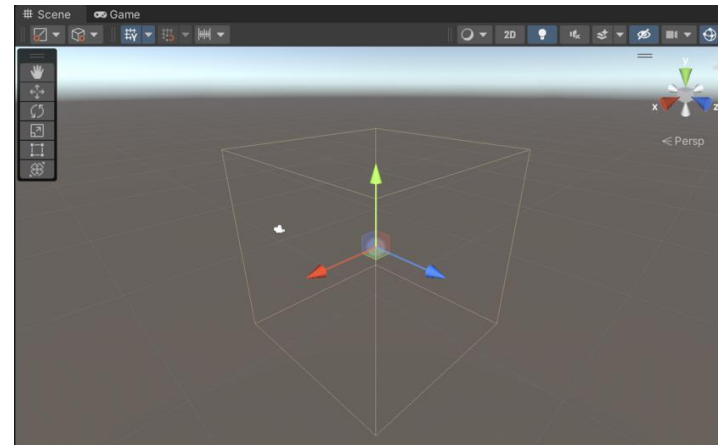
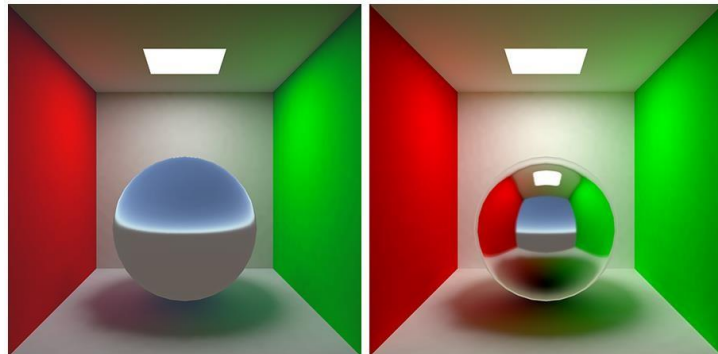
Reflections

- By default, **specular materials** reflect only the **Skybox**, making reflections appear uniform across the scene.
- However, in many cases, objects need to reflect their **immediate surroundings** for more realistic visuals.



Reflection Probes

- A **Reflection Probe** can be added via **GameObject** → **Light** → **Reflection Probe**.
- Used to capture **environment reflections** and apply them to objects for more realistic visuals.
- **Performance Tip:** Minimize the number of probes to optimize rendering.
- **Important:** Reflection probes provide **approximate reflections**, not physically accurate results.

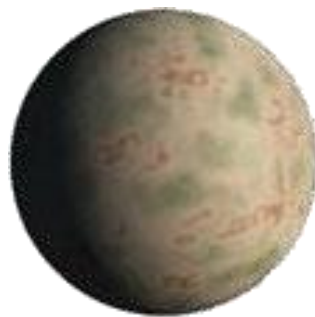


Bump Mapping



Bump Mapping (normal mapping)

- A **Bump Map** is a special texture that simulates **surface details** like bumps, grooves, and scratches **without adding extra geometry**.
- It affects how light interacts with the surface, creating the illusion of depth and texture.
- Commonly used to enhance realism while keeping models lightweight for performance optimization.



Base
Model

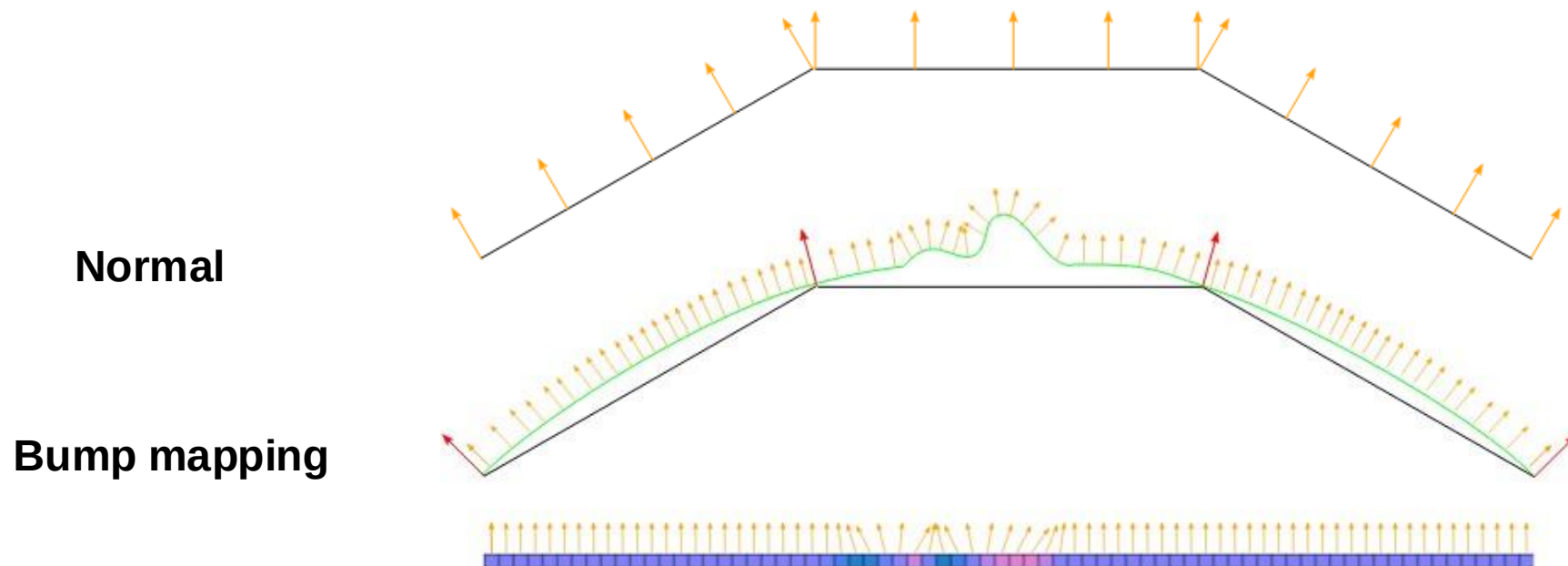


Bump
Mapping



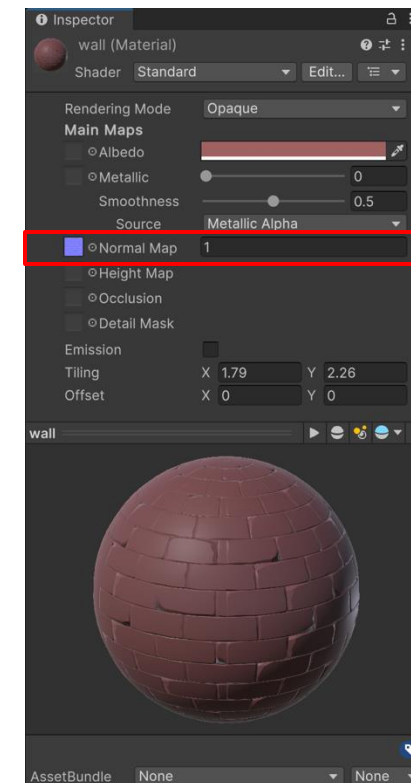
How bump maps work

- Each pixel in the **normal map** (called a **texel**) encodes a **deviation** in the surface's normal direction.
- This deviation alters how light interacts with the surface, creating the illusion of **depth and detail** on a flat or smoothly interpolated polygon.
- The actual geometry remains unchanged, but shading variations make the surface appear more complex.



Normal Maps

- **Created by 3D artists**, normal maps encode surface details to enhance lighting effects.
- They use **RGB data** to store **surface normal vectors**, guiding how light interacts with the surface.
- The **blue-purple appearance** is simply a **visual representation** in RGB space, not an actual material colour.
- **Different from Height Maps** → Normal maps affect **lighting** but do not physically displace geometry.



Heightmap

- A **Height Map** is a **grayscale texture** where brightness values represent height differences on a surface.
- While normal maps modify the lighting across the surface of the texture, height maps go a step further and actually shifts the areas of the visible surface texture around, to achieve a kind of surface-level occlusion effect.



Albedo, NormalMapping and Heightmapping

Albedo



Normalmap



Heightmap



IMPORTANT: in any case, none of the mapping alters the geometry of the mesh!!

Occlusion Map

- An **Occlusion Map** is a **grayscale texture** that controls how much **indirect lighting** a surface receives.
- **White** areas → Receive **full indirect lighting** (fully exposed).
- **Black** areas → Receive **no indirect lighting** (occluded or shadowed regions).
- Typically **created by 3D artists**, as accurate occlusion data can be difficult to generate automatically.



Original model



With ambient occlusion

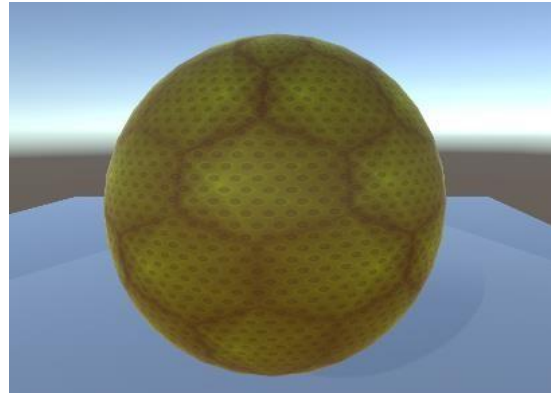


Extracted ambient occlusion map

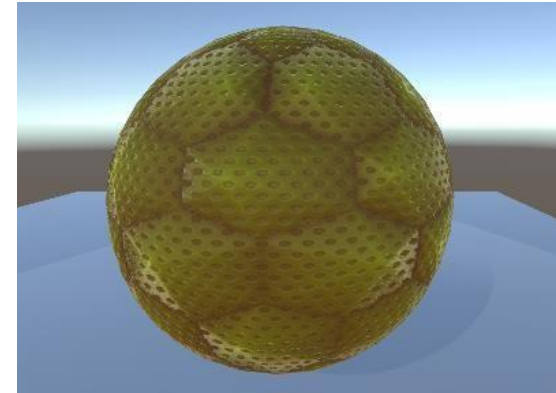
Resuming



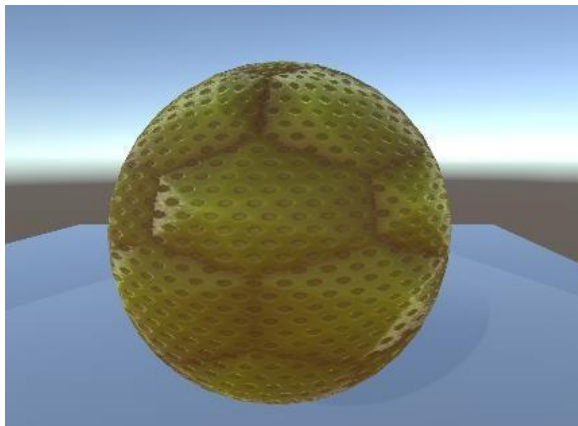
Albedo Texture



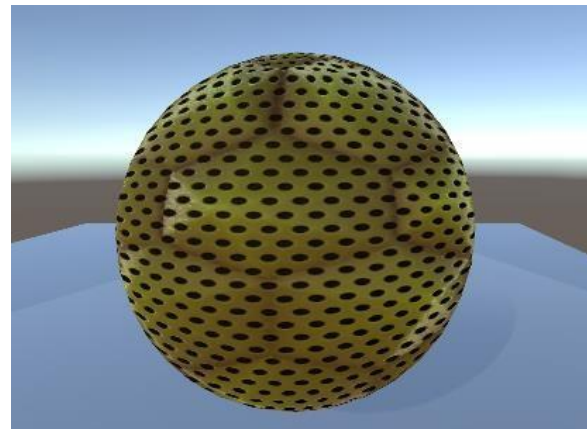
Normal Map



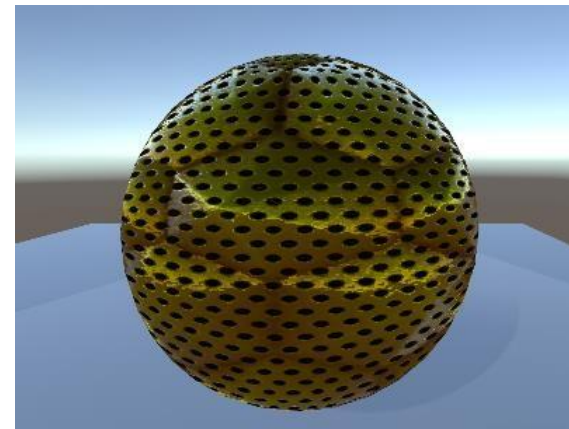
Height Map



Occlusion Map



Metallic Map



Emission



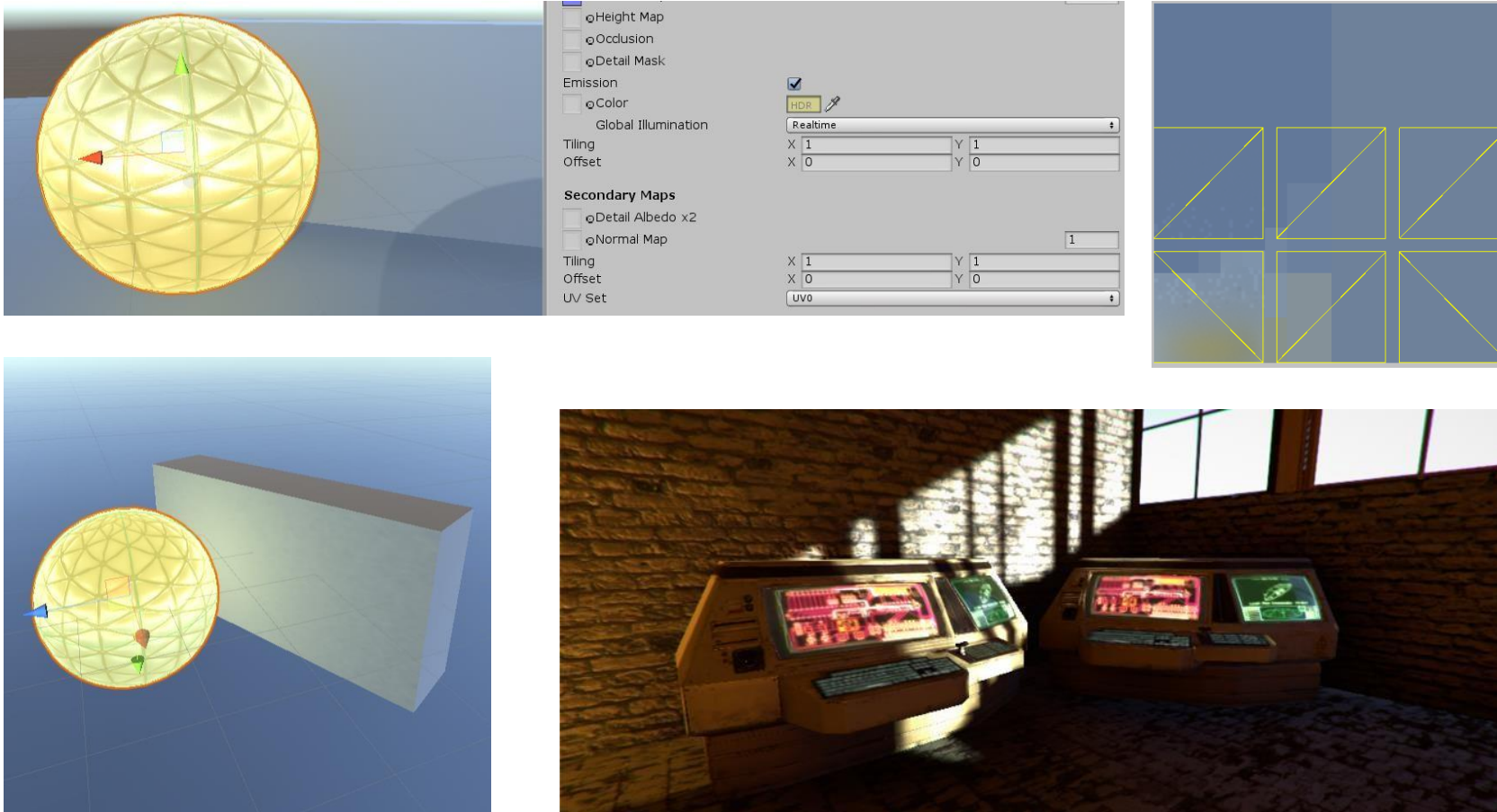
Emission

- Emissive materials are usually used on objects where some part should appear to be lit up from inside, such as the screen of a monitor, the disc brakes of a car braking at high speed, glowing buttons on a control panel, or a monster's eyes which are visible in the dark.
- Basic emissive materials can be defined using a **single colour** and an **emission intensity value**.
- If a texture map is assigned, the full colour values of the texture are used for the emission colour and brightness.

Emission materials enhance realism by making objects glow **without relying on external lighting**.

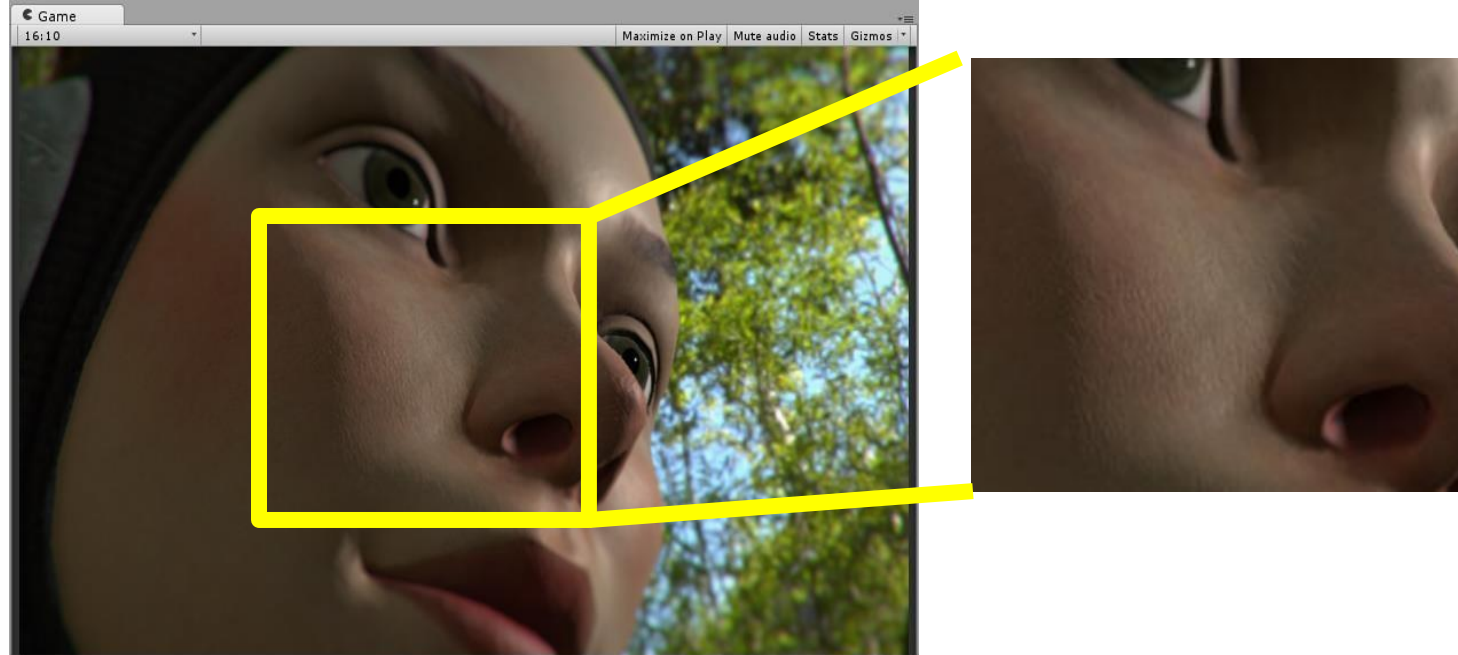
Emission

IMPORTANT: *Emissive materials reflect from Static on Static objects only (mixed lighting) or in realtime lighting.*



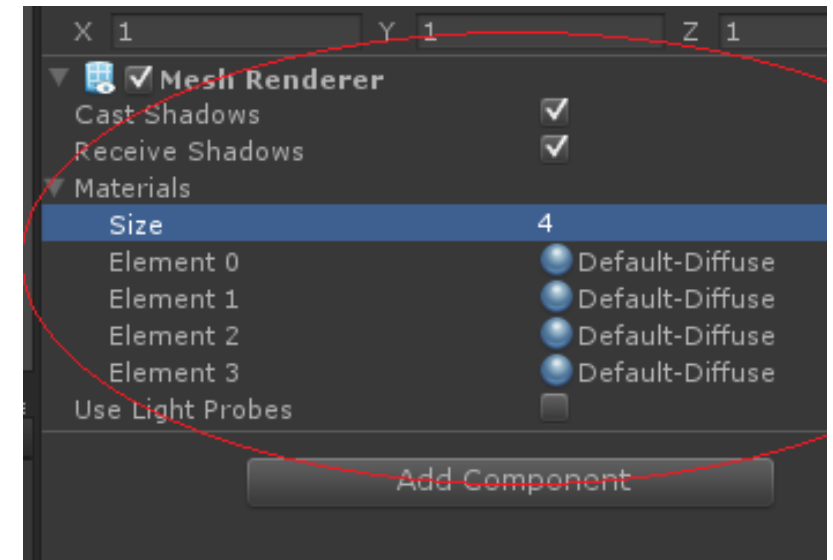
Secondary Maps

- **Secondary maps** add **extra detail** to materials, improving rendering quality.
- They help maintain **sharp details** when viewed up close while keeping a **normal level of detail** at a distance.
- This approach avoids the need for a **single extremely high-resolution texture**, optimizing **performance and memory usage**.



Multiple Materials

- A **single object** can have **multiple materials** applied to different parts of its mesh.
- The **Material option** in the **Mesh Renderer** allows you to assign and manage multiple materials.
- ⚠ **Performance Tip:** Using too many materials on a single object can negatively impact performance, so it's best to keep the material count minimal.



Saving and Exporting a Scene

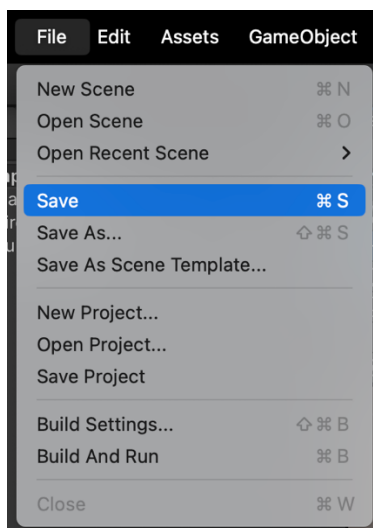


How to save

Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!

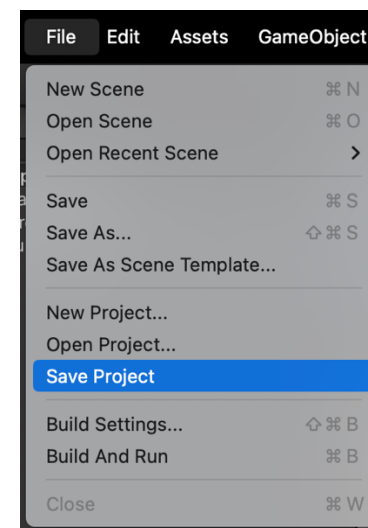
- Go to **File** → **Save** or press **Ctrl + S (Windows)** / **Cmd + S (Mac)**.
- This saves the current scene, including objects, lighting, and layout.



!! If you don't save the scene, any changes will be lost when Unity closes **!!**

✓ SAVE THE PROJECT!

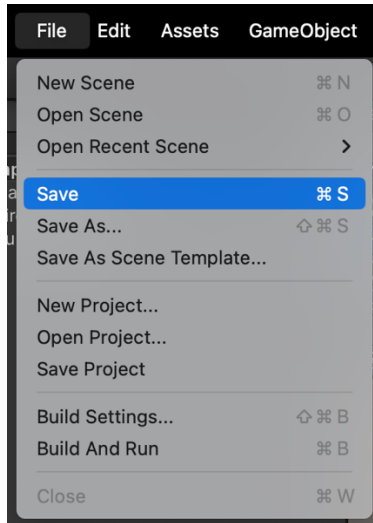
- Go to File → Save Project.
- This saves **project-wide settings**, such as imported assets, prefab modifications, and package settings.



How to save

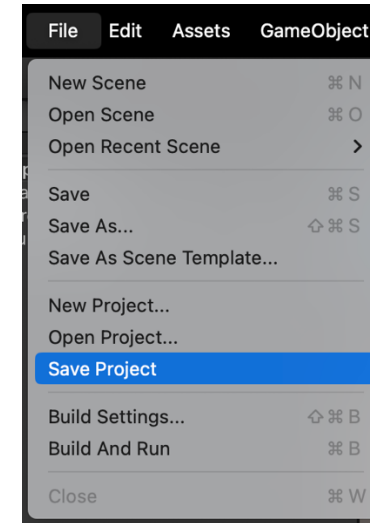
Unity stores different types of information about your project, and **not all changes are saved the same way**. It's important to understand the difference between **saving a scene** and **saving a project**.

✓ SAVE THE SCENE!



!! If you don't save the scene, any changes will be lost when Unity closes !!

✓ SAVE THE PROJECT!



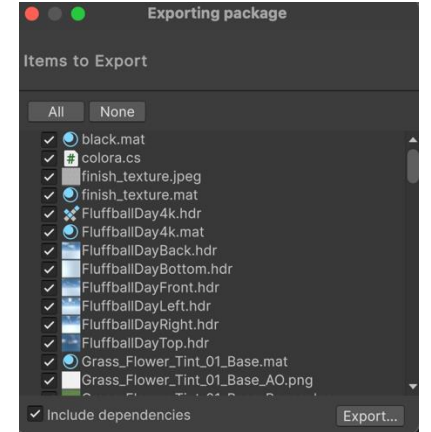
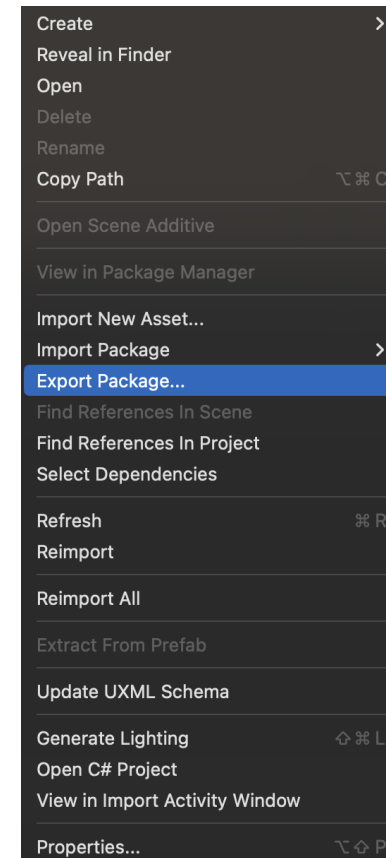
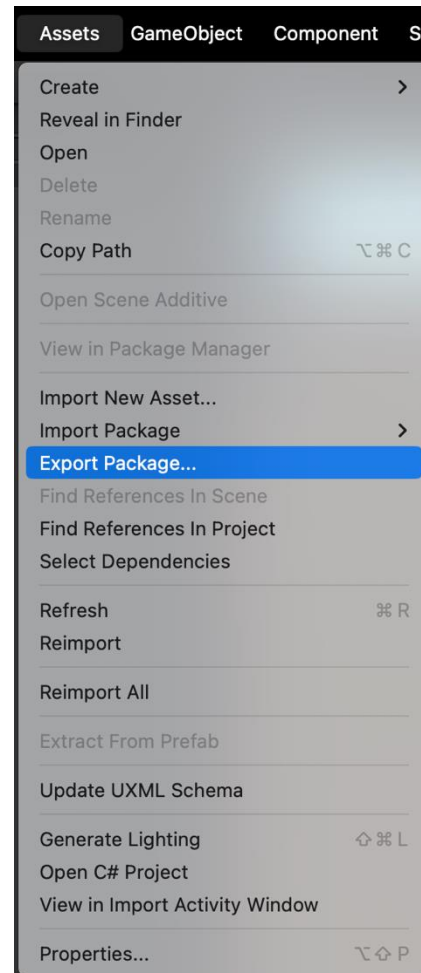
NOTE! Using “Save Project” **does not save** changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

Project Export



If you want to transfer your assets into a different project and preserve all the project's information, you can export your **assets** as a **Custom Package**.

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package...** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.



Project Export



WARNING: When exporting the project, make sure you are in the **correct folder**. To export the entire Assets folder, you must be inside the Assets folder. If you select a subfolder, only the elements within that subfolder will be exported, and the rest will not be exported!!!

1. Open the project you want to export assets from.
2. Choose **Assets > Export Package...** from the menu to bring up the **Exporting Package** dialog box.
3. In the dialog box, select the assets you want to include in the package by clicking on the boxes so they are checked.
4. Leave the **include dependencies** box checked to auto-select any assets used by the ones you have selected.
5. Click on **Export** to bring up File Explorer (Windows) or Finder (Mac) and choose where you want to store your package file. Name and save the package anywhere you like.

